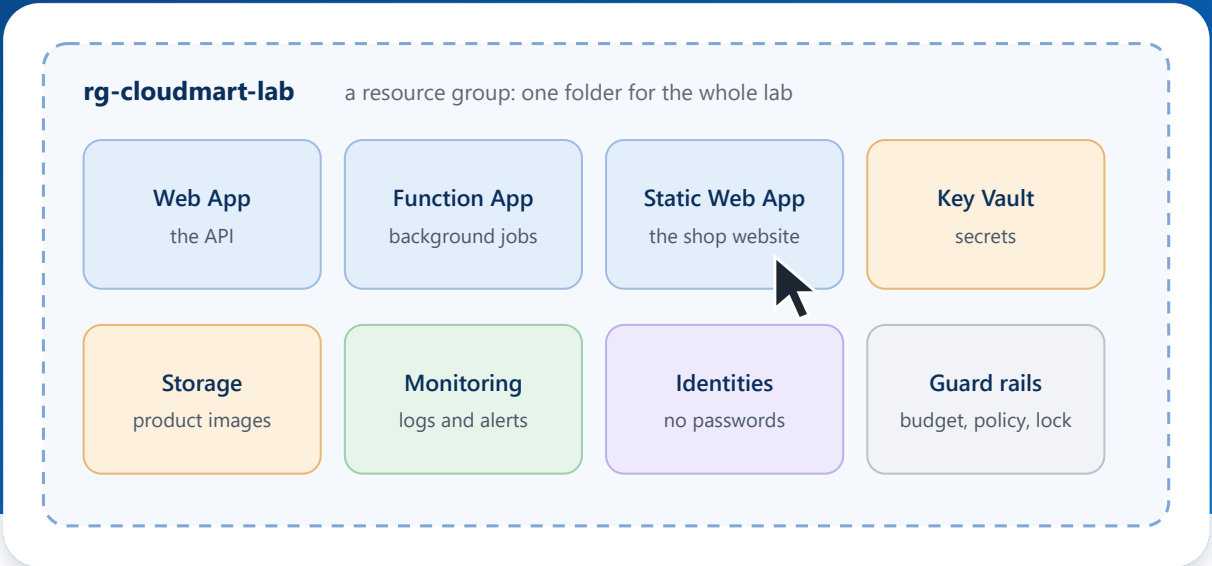
CLOUDMART · AZURE HANDS-ON SERIES · BEGINNER

Your first steps in the Azure Portal

Build CloudMart's foundation by clicking, one resource at a time. A visual guide for anyone who is new to Azure.



1 · Understand

Portal, CLI and Bicep, and the five ideas that make Azure make sense.

2 · Build

Thirteen illustrated steps that recreate what `infra/main.bicep` builds.

3 · Run

Put the CloudMart code on it and watch it work in the portal.

4 · Clean up

Connect clicks to code, delete everything, fix common errors.

How to use this guide

This guide is for someone who has never used Azure. You will build a small copy of CloudMart's cloud setup by clicking through the Azure Portal, so you can **see** every option before you automate it.

You will need

- A web browser and an **Azure subscription** where you are allowed to create things. A free account works (see Step 1).
- About **2 to 3 hours** for Part 2. You can stop between steps.
- An email address for budget and alert emails.
- For Part 3 only: a laptop with the .NET 10 SDK, Node.js 24, Git and the Azure CLI (CloudMart's `docs/learning-path.md`, phase 0).

The coloured boxes

WHY
What a thing is for, in plain words.

WATCH OUT
A mistake beginners often make.

CHECK IT WORKED
How to prove the step succeeded.

SAME THING IN BICEP
Where CloudMart's code does what you just clicked.

Click paths look like this: Search › Key vaults › + Create. Red numbers such as 1 point at parts of a picture. Values to type look like `this`.

Contents

Part 1 · Understand before you click

1.1 Portal, CLI or Bicep? A quick correction . . . 3
1.2 Azure in five ideas . . . 4
1.3 Two kinds of permission . . . 5
1.4 Money: avoiding a surprise bill . . . 6
1.5 A tour of the Azure Portal . . . 7
1.6 The pattern every "Create" follows . . . 8

Part 2 · Build CloudMart's foundation

What you will build . . . 9
Step 1 · Sign in and check your subscription . . . 11
Step 2 · Resource group . . . 12
Step 3 · Budget . . . 13
Step 4 · Log Analytics workspace . . . 14
Step 5 · Application Insights . . . 14
Step 6 · Two managed identities . . . 15
Step 7 · Key Vault . . . 16
Step 8 · Storage account . . . 19
Step 9 · Static Web App . . . 21
Step 10 · Web App for the API . . . 22
Step 11 · Function App . . . 24
Step 12 · An alert that emails you . . . 26
Step 13 · Guard rails: policy and lock . . . 27

Part 3 · Put CloudMart's code on it

Deploy, try the shop, watch it live . . . 28

Part 4 · From clicks to code

Export template, activity log, deployments . . . 30
Portal ↔ CLI ↔ Bicep cheat sheet . . . 31
Now do it the real way, with Bicep . . . 32

Part 5 · Clean up and reference

Clean up: delete everything . . . 33
Troubleshooting: messages and fixes . . . 34
Glossary . . . 35
What's next . . . 36

PART 1

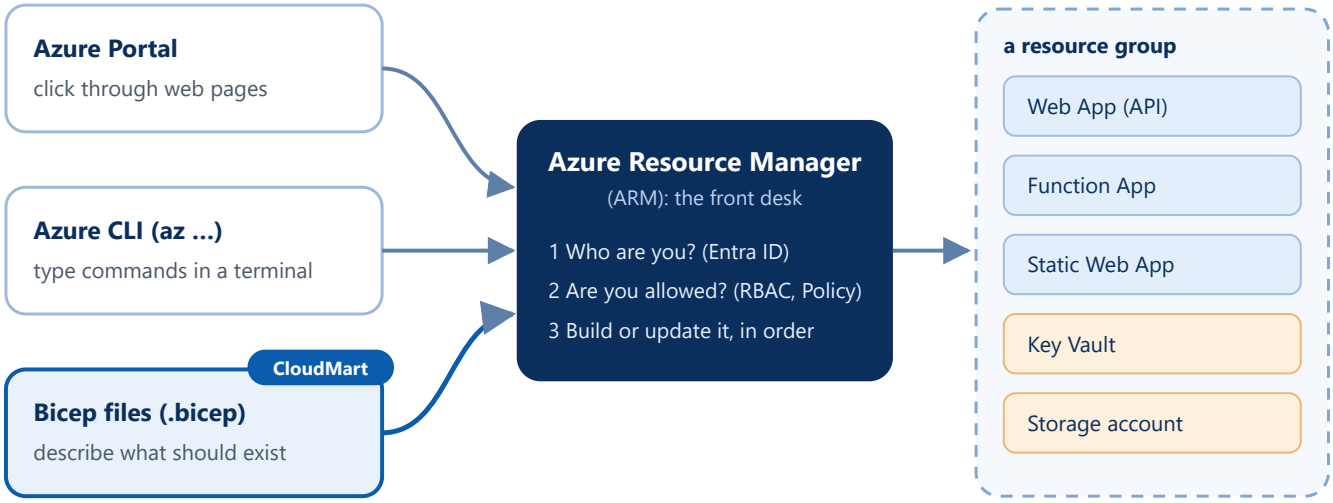
Understand before you click

Six short pages that make every later step easier.

1.1 Portal, CLI or Bicep? A quick correction

You may have heard that CloudMart "uses the Azure CLI to create its resources". That is **almost** right. Here is the precise picture:

- CloudMart's resources are **described in Bicep files**: `infra/main.bicep` plus 33 modules (and a shared `types.bicep`) in `infra/modules/`. Bicep is a blueprint. It says *what should exist*, not the clicks or commands to get there.
- The **Azure CLI is the courier**. `az deployment sub what-if` previews the blueprint and `az deployment sub create` hands it to Azure, which builds everything in the right order. The CLI does not create the resources one command at a time.
- The CLI is used directly only for hands-on jobs: signing in (`az login`), uploading code (`az webapp deploy`), building images (`az acr build`) and lab exercises.
- From phase 8, an **Azure Pipeline** runs the same commands for you whenever code is pushed.



Three doors, one room. Whatever you use, the request lands at Azure Resource Manager. A key vault made by a click is the same as one made by Bicep, which is why practising in the portal teaches you the Bicep.

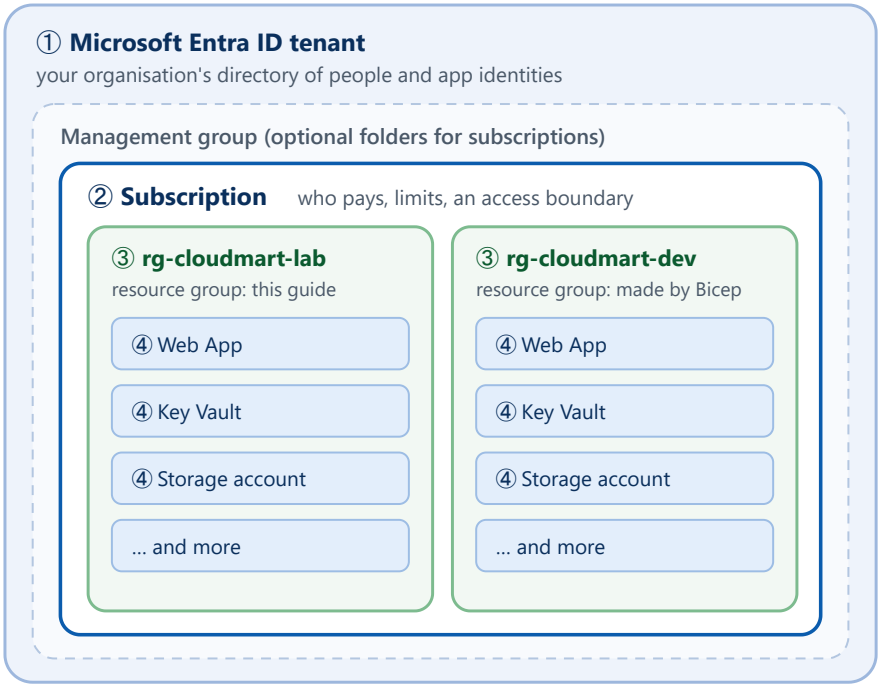
	Azure Portal	Azure CLI	Bicep
What it is	The website at portal.azure.com	A command-line tool, `az`	Text files that describe resources
Best for	Learning, exploring, checking, one-off fixes	Scripts, quick tasks, **launching** Bicep	Whole environments: repeatable, reviewed in Git
In CloudMart	You, while learning (this guide)	`az login`, `az deployment sub create`, `az webapp deploy`	`infra/`: the real source of truth

THE GOLDEN RULE: LEARN IN THE PORTAL, BUILD WITH BICEP

If you change a Bicep-managed resource by clicking, the next Bicep deployment may quietly put it back. This is called **drift**. So in this guide you build a **separate practice copy** in its own resource group, `rg-cloudmart-lab`, and delete it at the end. Your real Bicep environment, `rg-cloudmart-dev`, never sees your clicks.

1.2 Azure in five ideas

Almost everything in Azure fits into one picture. Learn these five words and the portal starts to make sense.



In everyday words

① **Tenant**
the company and its staff list

② **Subscription**
the company card that pays

③ **Resource group**
a project folder; delete the folder and everything inside goes

④ **Resource**
one thing in the folder: a web app, a vault, a database

⑤ **Region**
the city whose datacentres run it, e.g. East US 2

Three more facts that save beginners hours

- **Some names are global.** A key vault, storage account or web app name becomes a web address (for example `kv-cloudmart-xy123.vault.azure.net`), so nobody else in the world can be using it. That is why this guide adds a personal suffix, `xy123`. Replace it with **your initials and three random digits**, the same everywhere.
- **Resources live in one region**, chosen when you create them. Keep everything in the same region (this guide uses **East US 2**, like CloudMart's Bicep) so the parts talk to each other quickly and cheaply.
- **Tags are sticky notes** (name = value) you put on resources, for example `environment = lab`. Cost reports and policies can use them.

> **BEGINNER NOTE: "RESOURCE PROVIDER"**
> Each kind of resource is run by a **resource provider**, such as `Microsoft.KeyVault` or `Microsoft.Web`. A provider must be **registered** (switched on) for your subscription before you can use it. Azure usually does this for you; Step 1 shows what to do when it doesn't.

1.3 Two kinds of permission

This is the idea that surprises almost every beginner, and you will meet it in Steps 7 and 8. Azure has **two separate sets of locks**:

CONTROL PLANE

Managing the resource

like holding the keys to the building

- create, configure and delete it
- change settings, tags, network rules
- give other people access

Roles: Owner, Contributor, Reader
granted on a subscription, resource group or resource

DATA PLANE

Using what is inside it

like the combination to the safe inside

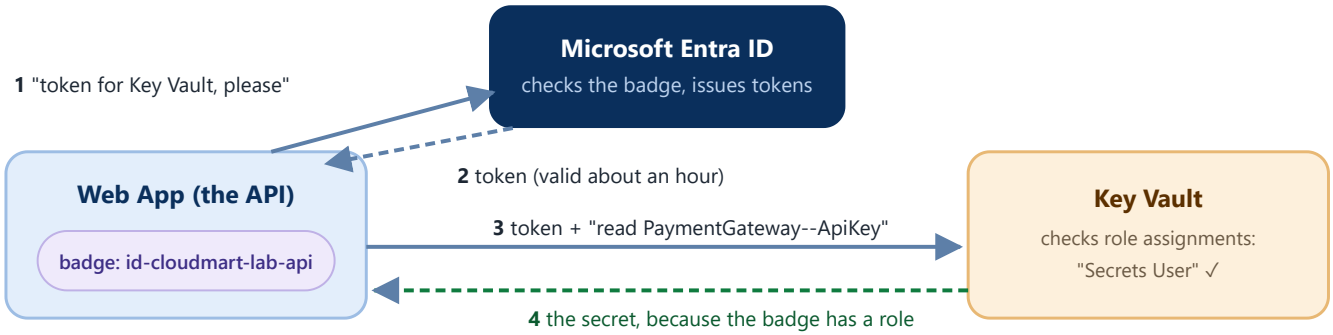
- read or write secrets in a key vault
- upload or download files (blobs)
- read queues, tables, database rows

Roles: Key Vault Secrets User / Officer, Storage Blob Data Contributor, ...

You, as Owner of the subscription: ✓ can create the vault ✗ cannot read its secrets
... until you also give yourself a data role (Step 7). Owner does not include the data plane.

Managed identities: an ID badge for an app

Apps need data roles too. Instead of giving an app a password or key (which can leak), Azure gives it a **managed identity**: a badge that Microsoft Entra ID recognises. The app shows the badge, gets a short-lived **token**, and the vault checks whether that badge has a role. CloudMart uses no passwords or keys at all, only identities.



No password or key is stored anywhere in the app's code or settings.

SYSTEM-ASSIGNED OR USER-ASSIGNED?

A **system-assigned** identity is born and dies with one app. A **user-assigned** identity is its own resource: you create it first, give it roles, then attach it to any app. CloudMart uses user-assigned identities (`infra/modules/identity.bicep`), so you will too.

1.4 Money: avoiding a surprise bill

Azure charges in three main ways: **per hour** for things that are always running (like an App Service plan), **per use** for serverless things (Functions, storage transactions) and **per gigabyte** for stored data and logs. Here is what the lab costs at list prices (check the pricing calculator for your region):

Resource in this lab	How it is charged	Rough cost
App Service plan, Basic B1	Per hour while it exists, even if idle	≈ **US$13 a month** (≈ 2 cents an hour). The main cost.
Function App, Flex Consumption	Per execution and memory used	≈ $0 while idle
Static Web App, Free plan	Free	$0
Storage account (LRS)	Per GB stored and per operation	a few cents
Key Vault (Standard)	Per 10,000 operations	a few cents
Log Analytics + Application Insights	Per GB of logs collected	cents at lab volume; capped at 1 GB a day in Step 4
Log search alert rule	Per rule per month	≈ $1.50 a month (pro-rated)
Resource group, identities, budget, policy, lock	Free	$0

THE BOTTOM LINE

If you do Part 2 and Part 3 in one sitting and delete everything in Part 5, expect **well under US$1**. Leave it running for a month and it is roughly US$15.

Three habits that protect you

1. **Budget first**
   Create a budget with email alerts before anything else (Step 3).
2. **Delete when done**
   Delete the whole resource group at the end (Part 5).
3. **Look the next day**
   Open Cost Management and check that spending stopped.

WATCH OUT: A BUDGET IS AN ALARM, NOT A LIMIT

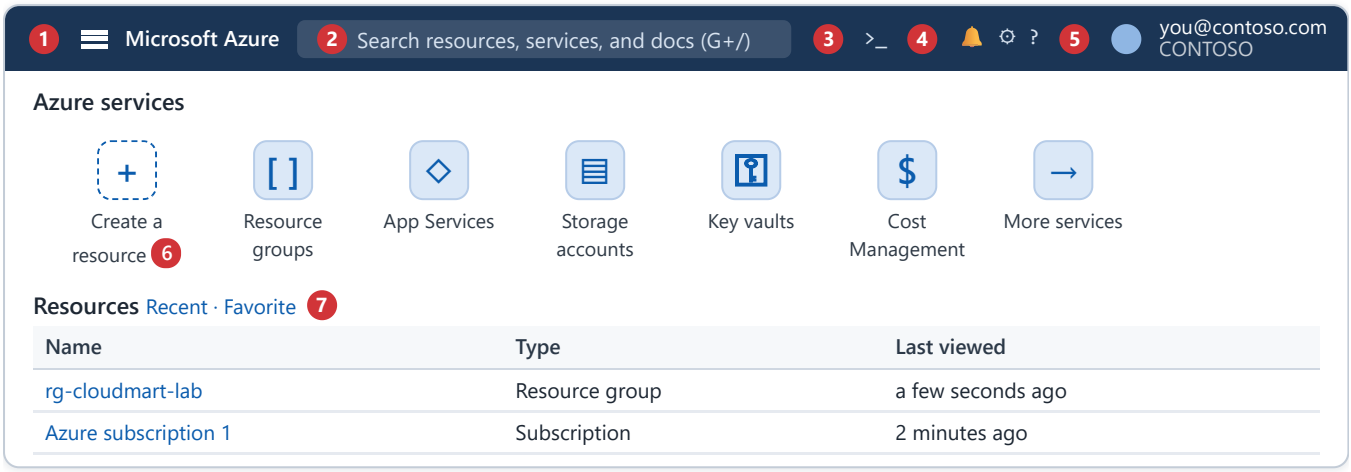
An Azure budget **emails** you when spending crosses a line. It does **not** stop or delete anything. Only deleting resources stops the charges. Cost data also updates a few times a day, not instantly.

FREE ACCOUNT

A new Azure free account usually comes with some credit for the first 30 days plus always-free amounts of popular services. Check the current offer at azure.microsoft.com/free. At work, ask your Azure administrator for a sandbox subscription instead of using a production one.

1.5 A tour of the Azure Portal

Open **https://portal.azure.com** and sign in. The home page looks roughly like this:



Simplified illustration of the portal home page.

1 **Portal menu**: Home, Dashboard, All services, Favorites.

2 **Search bar**: the fastest way anywhere. Type a service ("key vaults") or a resource's name.

3 **Cloud Shell** (>_): a terminal in your browser with the Azure CLI and Bicep already installed.

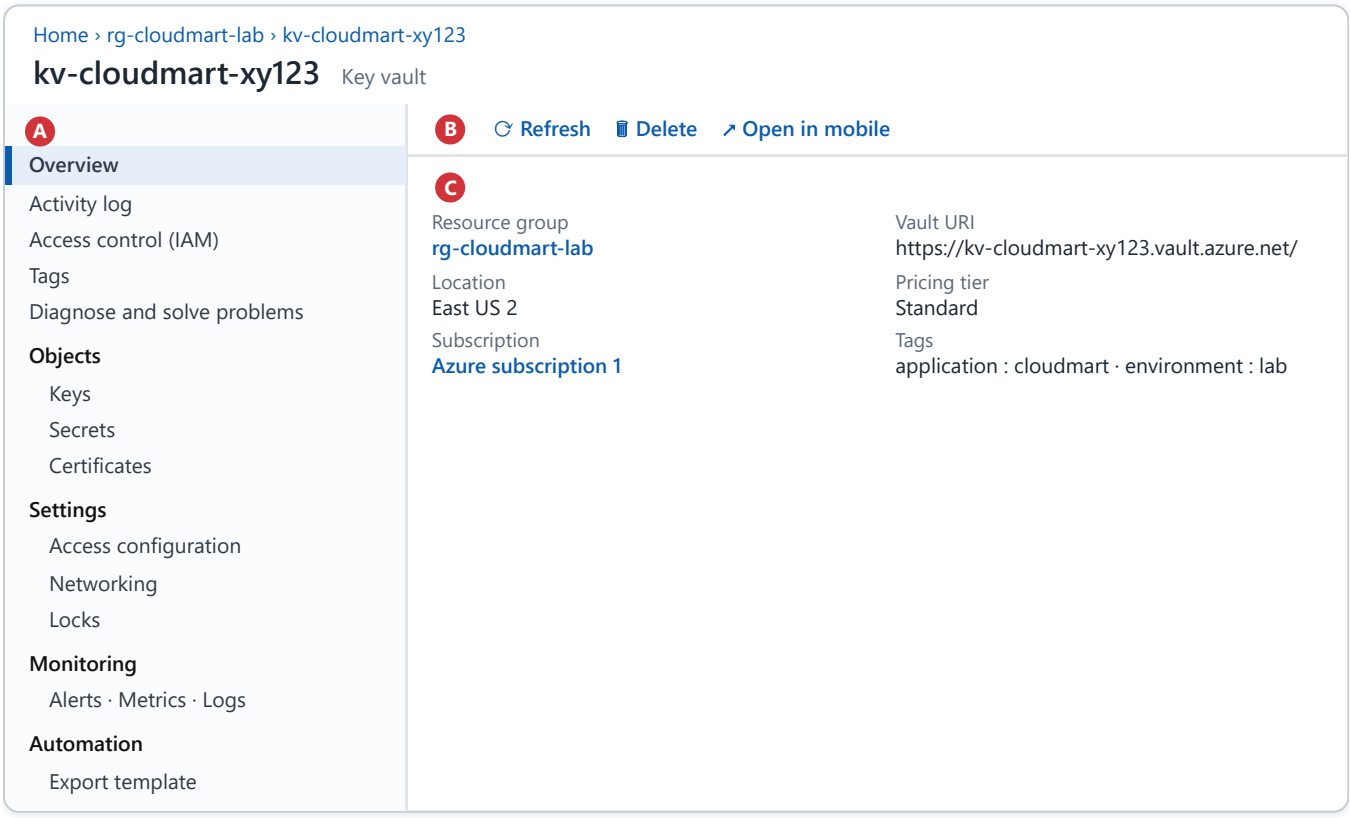
4 **Notifications** (bell): deployment progress appears here.

5 **Settings, help, account**: switch directory (tenant), theme, language; see who you are signed in as.

6 **Create a resource**: the Marketplace of everything you can create.

7 **Recent resources**: jump back to what you were working on.

Every resource page has the same skeleton



Simplified illustration of a resource page (Microsoft calls these pages **blades**).

A **Left menu**. Every resource has Overview, Activity log, Access control (IAM), Tags and, under Settings, Locks.

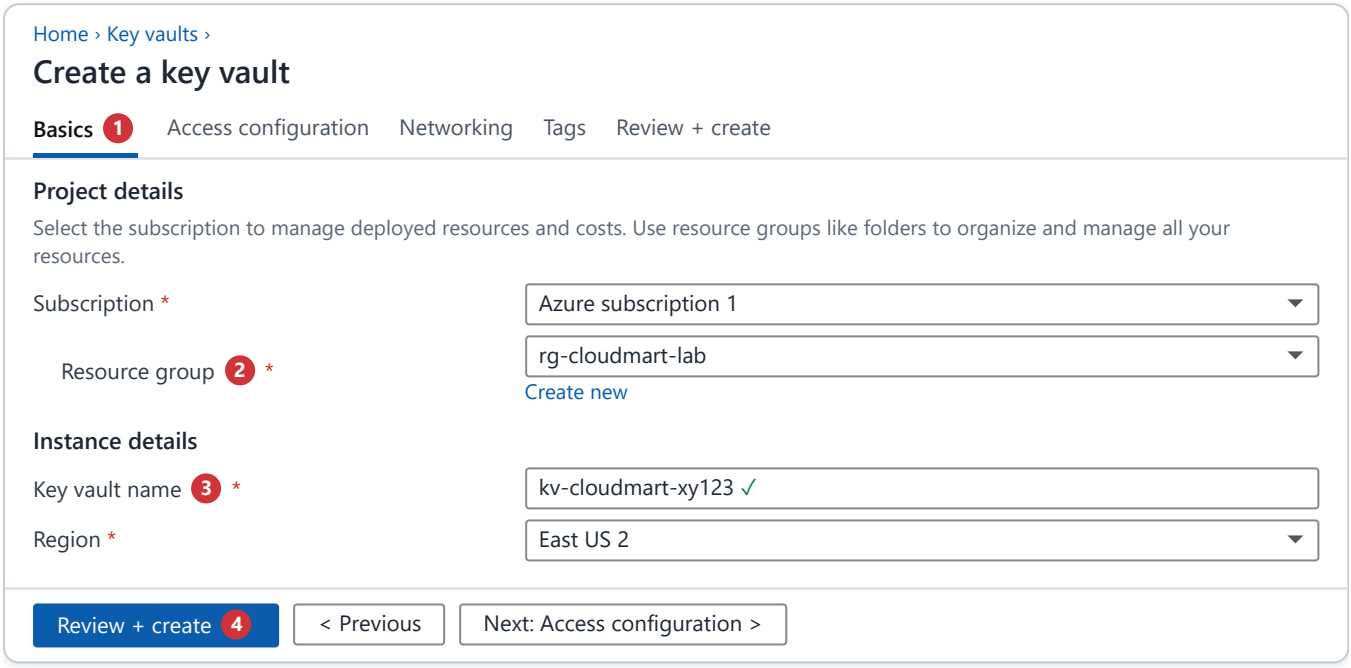
B **Command bar**: actions for this resource, such as Delete or Restart.

C **Essentials**: the key facts (group, region, subscription, address). You will copy values from here.

1.6 The pattern every "Create" follows

Creating anything in the portal, from a resource group to a database, goes through the same wizard. Once you know the pattern, every step in Part 2 feels familiar.

1. **Find it** — Search the service, e.g. "Key vaults".
2. **+ Create** — Opens the create wizard.
3. **Basics tab** — Subscription, resource group, name, region, size.
4. **Other tabs** — Networking, security, monitoring, tags.
5. **Review + create** — Azure validates your choices.
6. **Create** — Sends the request to Resource Manager.
7. **Deployment in progress** — Watch it under the bell.
8. **Go to resource** — Finish the settings there.



Simplified illustration of a create wizard.

1 **Tabs** run left to right. You can jump between them; most have sensible defaults.

2 **Resource group**: always pick `rg-cloudmart-lab` in this guide.

3 **Name**: a green tick means it is valid and free. A red asterisk (*) means required.

4 **Review + create** runs validation. You must then click **Create** on the next page.

WHY THE REVIEW PAGE MATTERS

The Review + create page usually has a link such as **Download a template for automation** (or **View automation template**). It shows the exact template the portal is about to send, which is the same kind of file that CloudMart's Bicep produces. Peek at it once; it makes Part 4 obvious.

HOW TO TELL A CREATE WORKED

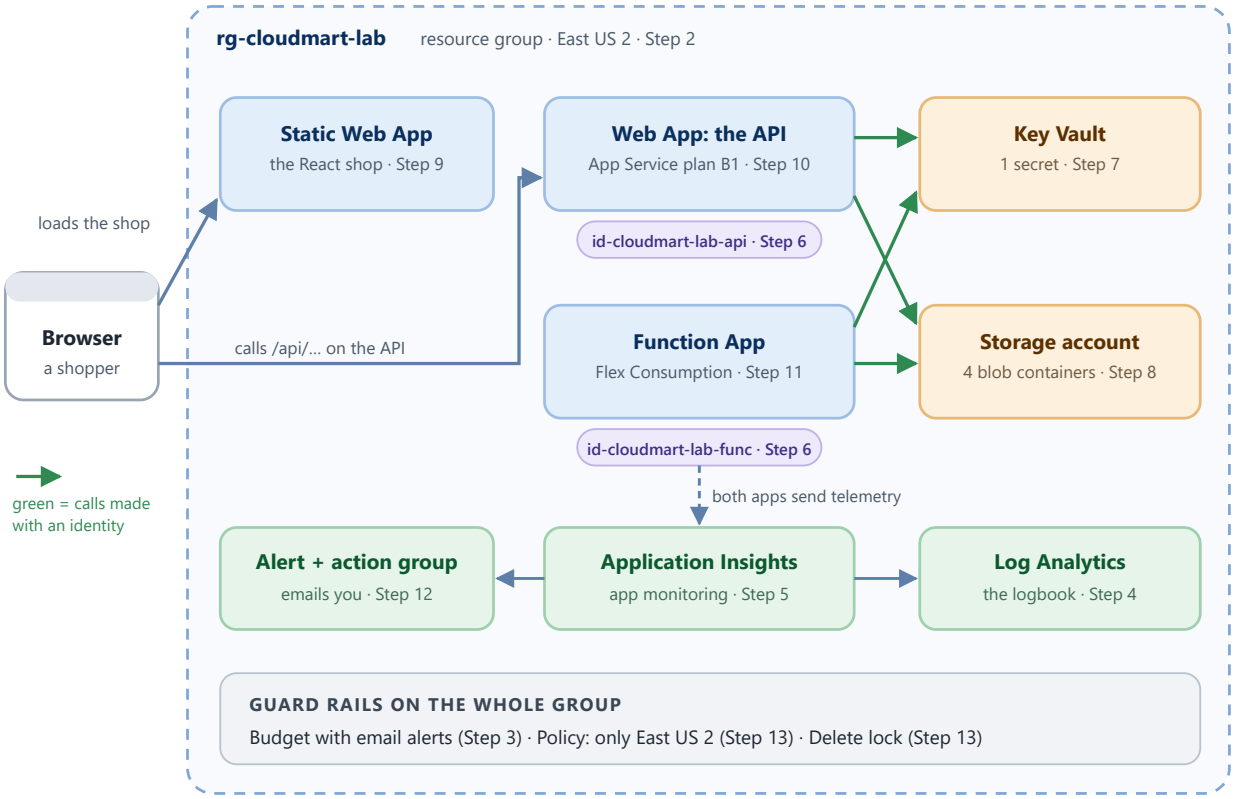
The page changes to **"Your deployment is complete"** with a green tick, and the bell shows "Deployment succeeded". Click **Go to resource**. If it fails, the page shows an error code; look it up in the Troubleshooting table at the back.

PART 2

Build CloudMart's foundation by hand

Thirteen steps that create, by clicking, what `infra/main.bicep` creates with its default switches (phase 1).

What you will build



The lab, step by step. Green arrows are calls made with a managed identity and a data role, never a key.

Names used in this guide

Replace `xy123` with your initials and three digits, and use the same suffix everywhere. The right-hand column shows what CloudMart's Bicep calls the same thing, so you can recognise it later in `rg-cloudmart-dev`.

Step	Resource	Name in this lab	Name made by Bicep
2	Resource group	`rg-cloudmart-lab`	`rg-cloudmart-dev`
4	Log Analytics workspace	`log-cloudmart-lab`	`log-cloudmart-dev`
5	Application Insights	`appi-cloudmart-lab`	`appi-cloudmart-dev`
6	Managed identities	`id-cloudmart-lab-api`, `-func`	`id-cloudmart-dev-api`, `-func` (+2)
7	Key Vault	`kv-cloudmart-xy123`	`kv-cloudmart-` + generated
8	Storage account	`stcloudmartxy123`	`stcloudmart` + generated characters
9	Static Web App	`stapp-cloudmart-lab`	`stapp-cloudmart-dev`
10	App Service plan	`asp-cloudmart-lab`	`asp-cloudmart-dev`
10	Web App (API)	`app-cloudmart-lab-api-xy123`	`app-cloudmart-dev-api-` …
11	Function App	`func-cloudmart-lab-xy123`	`func-cloudmart-dev-` …
12	Action group	`ag-cloudmart-lab`	`ag-cloudmart-dev`

Your lab notebook

Several steps ask you to copy a value now and paste it later. Write them here (or in a text file) as you go. None of these values are secret, except where noted.

Value	Where you find it	Write it here
Your suffix (initials + 3 digits)	you choose	
Subscription name	Step 1	
Application Insights connection string	Step 5, Overview	
Client ID of id-cloudmart-lab-api	Step 6, Overview	
Client ID of id-cloudmart-lab-func	Step 6, Overview	
Key Vault URI	Step 7, Overview	
Storage blob endpoint	Step 8, Endpoints	
Static Web App URL	Step 9, Overview	
Web App (API) default domain	Step 10, Overview	
Function App name	Step 11	

Your progress

	Step	Time	You'll learn
☐	1 · Sign in, check subscription	5 min	Subscriptions, your role, resource providers
☐	2 · Resource group	3 min	The create pattern, tags
☐	3 · Budget	5 min	Cost alerts
☐	4 · Log Analytics workspace	5 min	Central logging, daily cap
☐	5 · Application Insights	3 min	App monitoring
☐	6 · Two managed identities	5 min	Identity without passwords
☐	7 · Key Vault	15 min	Secrets, control vs data plane, role assignments
☐	8 · Storage account	15 min	Blob storage, turning keys off, data roles
☐	9 · Static Web App	5 min	Hosting a website, CORS
☐	10 · Web App for the API	15 min	App Service, app settings, identities on apps
☐	11 · Function App	10 min	Serverless, Flex Consumption
☐	12 · Alert	10 min	Log search alerts, action groups
☐	13 · Policy and lock	10 min	Governance: what may exist, what may be deleted

STEP 1

Sign in and check your subscription

TIME 5 min **COST** free

WHY

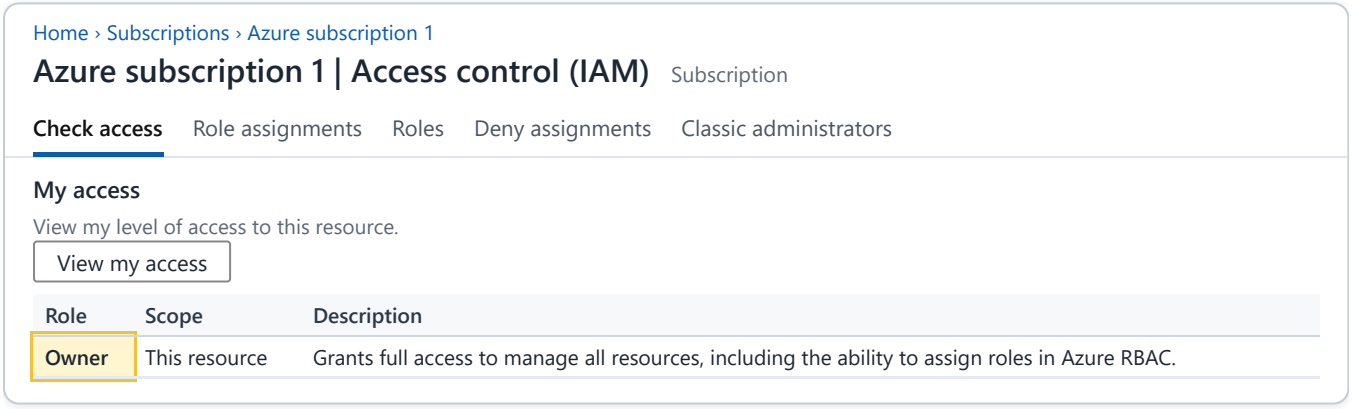
Everything you create is billed to a subscription, and later steps assign roles to identities. You need to know which subscription you are in and that you are allowed to do both.

1. Go to **https://portal.azure.com** and sign in with the account that owns (or was given) the subscription.
2. Type **Subscriptions** in the search bar and open it. You should see at least one subscription with status **Active**. Write its name in your notebook.

   YOU SEE NONE?

   You have no subscription in this directory. Create a free account at azure.microsoft.com/free, or use the **Settings** (gear) icon to switch to another directory. At work, ask your Azure administrator for a sandbox.
3. Click the subscription, then **Access control (IAM)** › **Check access** › **View my access**. You need **Owner**, or **Contributor** plus **Role Based Access Control Administrator** (or User Access Administrator), because Steps 7 and 8 assign roles.
4. Still in the subscription, open **Settings** › **Resource providers**. You do not need to change anything now; just know where this page is.

CLICK PATH **Search** › **Subscriptions** › **your subscription** › **Access control (IAM)** › **Check access** › **View my access**



Simplified illustration. **Owner** at subscription scope is what you want for this lab.

WATCH OUT: "THE SUBSCRIPTION IS NOT REGISTERED TO USE NAMESPACE …"

If a later step fails with this message, come back to **Resource providers**, search for the namespace it names (for example `Microsoft.Web`), select it and click **Register**. Wait until it says **Registered** (a minute or two), then retry.

CHECK IT WORKED

You know your subscription's name and your role on it is Owner (or equivalent).

STEP 2

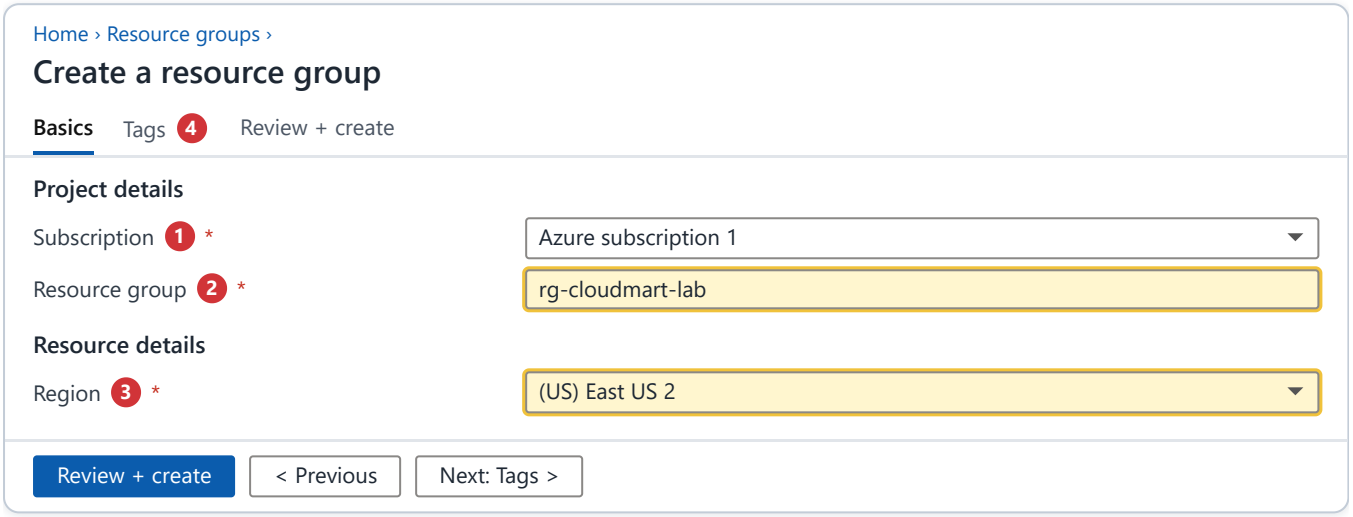
Create the resource group

TIME 3 min · **COST** free · **BICEP** infra/main.bicep (resource rg)

WHY

A resource group is the folder for this whole lab. Everything in Part 2 goes inside it, and in Part 5 you delete the folder to delete everything at once.

CLICK PATH **Search** › **Resource groups** › **+ Create**



Simplified illustration. Highlighted fields are the ones you change.

Field	Enter or choose	Why
1 **Subscription**	your subscription	Who pays for what goes in this group.
2 **Resource group**	`rg-cloudmart-lab`	"rg-" is the naming prefix Microsoft's Cloud Adoption Framework recommends; CloudMart uses it too.
3 **Region**	`East US 2`	Where the group's own record lives. Keep every resource in the same region.

4 **Tags tab**	`application = cloudmart`
	`environment = lab`
	`managed-by = portal`
		Labels for cost reports and policy. Bicep sets the same tags, with `managed-by = bicep`.

Click **Review + create**, wait for **Validation passed**, then click **Create**. A resource group is created in a second or two.

CHECK IT WORKED

Open **Resource groups**: `rg-cloudmart-lab` is listed. Open it: it says there are no resources yet, and the **Tags** on its Overview show your three tags.

SAME THING IN BICEP

```
resource rg 'Microsoft.Resources/resourceGroups@2024-03-01' = {
  name: 'rg-cloudmart-dev'    // names.resourceGroup
  location: 'eastus2'
  tags: { application: 'cloudmart', environment: 'dev', 'managed-by': 'bicep' }
}
```

STEP 3

Set a budget before you spend anything

TIME 5 min **COST** free **BICEP** infra/modules/governance.bicep

WHY

A budget emails you when spending on this resource group reaches a level you choose. It is the cheapest insurance in Azure. Remember: it warns, it does not stop anything.

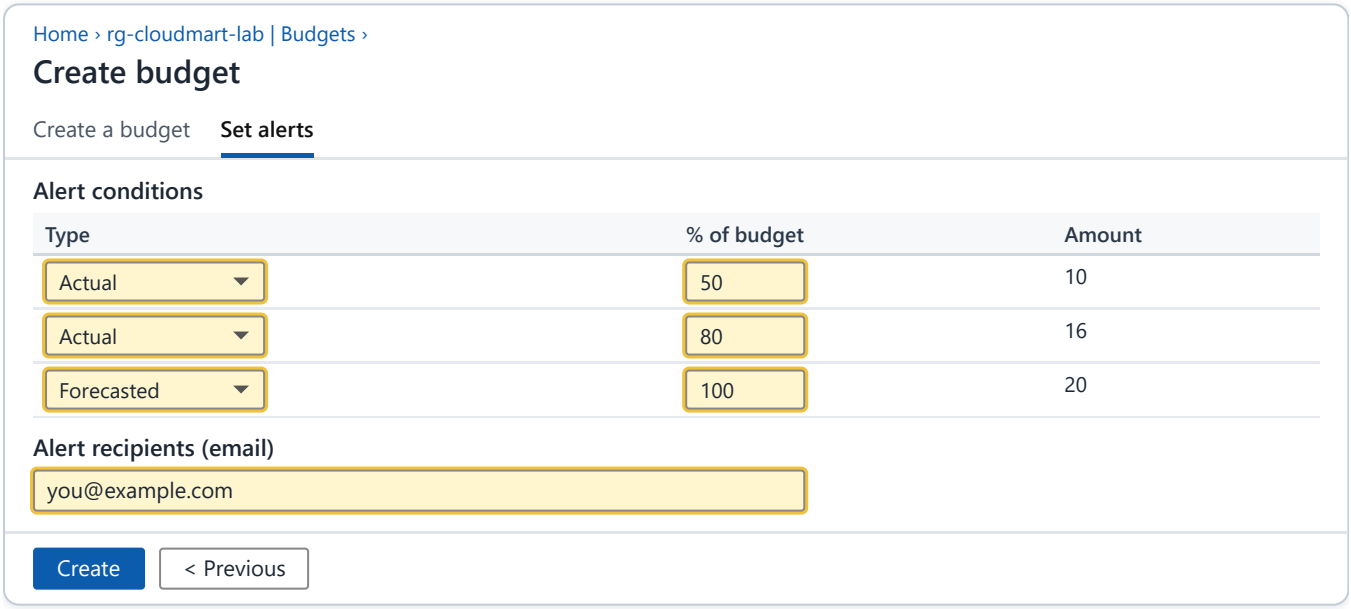
CLICK PATH **rg-cloudmart-lab** › **Budgets** (left menu, under Cost Management) › **+ Add**

Page 1: Create a budget

Field	Enter or choose	Why
Scope	shows `rg-cloudmart-lab`	The budget only counts costs inside this group.
Name	`cloudmart-lab-monthly`	Bicep's budget is called `cloudmart-monthly`.
Reset period	`Monthly`	Starts counting from zero every month.
Creation / expiration date	leave the defaults	
Amount	`20`	In your billing currency. CloudMart's Bicep uses 50 for the full lab.

Click **Next**.

Page 2: Set alerts



Simplified illustration. "Forecasted" warns you early: when Azure predicts you will reach 100% by the end of the month.

Click **Create**.

CHECK IT WORKED

Budgets now lists `cloudmart-lab-monthly` with an amount of 20 and current spend of 0.

SAME THING IN BICEP

`governance.bicep` creates the same three alerts: `actual50`, `actual80` and `forecast100`, sent to `ownerEmail` from `main.bicepparam`.

STEP 4

Log Analytics workspace: the logbook

TIME 5 min · **COST** per GB, capped · **BICEP** infra/modules/monitoring.bicep

WHY

One central place where every service writes its logs. You search it with a query language called **KQL**. Creating it first means everything you build later can send logs here from day one.

CLICK PATH Search › Log Analytics workspaces › + Create

Field	Enter or choose	Why
Resource group	`rg-cloudmart-lab`	Always this group.
Name	`log-cloudmart-lab`	"log-" prefix, as in Bicep.
Region	`East US 2`	
Tags	the same three tags as Step 2	Do this on every resource.

Review + create › **Create** › **Go to resource**. Then add a safety cap:

1. In the workspace's left menu open **Settings** › **Usage and estimated costs**.
2. Click **Daily cap**, set it **On** with `1` GB/day, and click **OK**. Leave **Data retention** at 30 days.

WATCH OUT

When the daily cap is reached, the workspace **stops collecting** until the next day. Great for a lab budget; risky in production, where you would rather pay than lose logs.

STEP 5

Application Insights: watching the app

TIME 3 min · **COST** per GB, shares the cap · **BICEP** infra/modules/monitoring.bicep

WHY

Application Insights records what the app does: each request, how long it took, failures, and which service called which. It stores everything inside the workspace from Step 4.

CLICK PATH Search › Application Insights › + Create

Field	Enter or choose	Why
Resource group · Name	`rg-cloudmart-lab` · `appi-cloudmart-lab`	"appi-" prefix, as in Bicep.
Region	`East US 2`	
Log Analytics workspace	`log-cloudmart-lab`	"Workspace-based" is now the only kind; older "classic" resources are retired.

Review + create › **Create** › **Go to resource**. On the **Overview** page, copy the **Connection string** into your notebook. You will paste it into the website's settings in Part 3; the Web App and Function App wizards link to it for you.

CHECK IT WORKED

The resource group now holds `log-cloudmart-lab` and `appi-cloudmart-lab`. (You may also see an alert rule called *Failure Anomalies* that Azure adds automatically.)

STEP 6

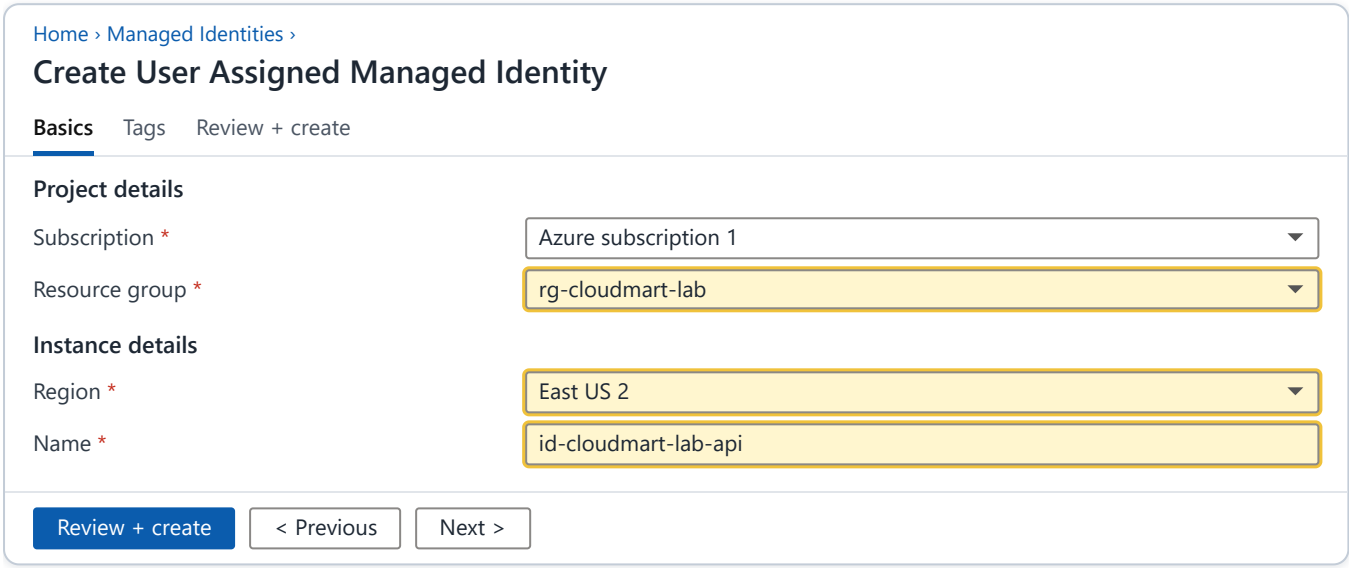
Two managed identities: ID badges for the apps

TIME 5 min **COST** free **BICEP** infra/modules/identity.bicep

WHY

Each app gets its own badge, so each can be given only the access it needs (the idea of **least privilege**). Creating the badges before the apps lets you grant access in Steps 7 and 8 before the apps even exist.

CLICK PATH **Search** › **Managed Identities** › **+ Create**



Simplified illustration. Do this twice.

1. Create `id-cloudmart-lab-api` (the badge for the API in Step 10). Add the usual tags.
2. Create `id-cloudmart-lab-func` (the badge for the Function App in Step 11).
3. Open each identity's **Overview** and copy its **Client ID** into your notebook. The app code uses the Client ID to say which badge it wants to wear.

CLIENT ID OR OBJECT (PRINCIPAL) ID?

An identity shows two IDs. The **Client ID** is what an app's settings use (`AZURE_CLIENT_ID`). The **Object (principal) ID** is what role assignments use behind the scenes. In the portal you pick identities by name, so you only need to note the Client ID.

CHECK IT WORKED

Both identities appear in `rg-cloudmart-lab` with type **Managed Identity**. Right now they can do nothing at all; Steps 7 and 8 give them roles.

SAME THING IN BICEP

```
resource api 'Microsoft.ManagedIdentity/userAssignedIdentities@2023-01-31' = {
  name: apiIdentityName      // id-cloudmart-dev-api
  location: location
  tags: tags
}
```

STEP 7

Key Vault: a safe for secrets

TIME 15 min **COST** cents **BICEP** infra/modules/keyvault.bicep

WHY

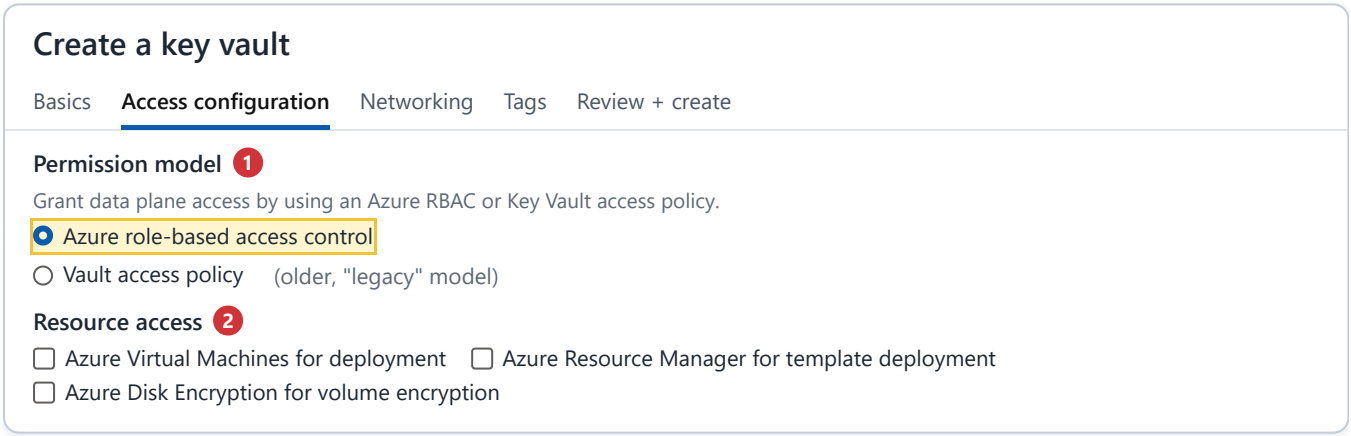
Key Vault keeps secrets (API keys, passwords, certificates) out of code and settings. CloudMart stores a pretend payment-gateway key in it to show how an app reads a secret with no password of its own.

7a · Create the vault

CLICK PATH Search › Key vaults › + Create

Field	Enter or choose	Why
Resource group	`rg-cloudmart-lab`	
Key vault name	`kv-cloudmart-xy123`	3–24 letters, digits and hyphens; must be unique worldwide because it becomes `https://kv-cloudmart-xy123.vault.azure.net`.
Region · Pricing tier	`East US 2` · `Standard`	Premium adds hardware-protected keys; not needed here.
Days to retain deleted vaults	`7`	Deleted vaults are kept ("soft-deleted") this long, then gone. 7 is the minimum.
Purge protection	`Disable`	Lab only. In production, enable it: then nobody can permanently delete the vault early.

Now open the **Access configuration** tab. This choice matters more than any other in this step:



Simplified illustration of the Access configuration tab.

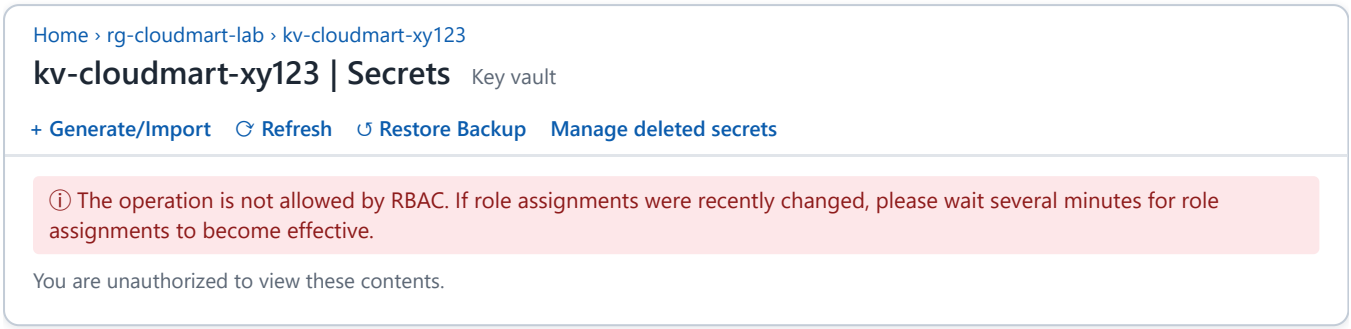
1 **Permission model: Azure role-based access control.** Access to secrets is then given with ordinary role assignments, the same way as everything else in Azure. CloudMart's Bicep sets `enableRbacAuthorization: true`.

2 **Resource access**: leave all three unticked; CloudMart doesn't need them.

On **Networking** leave public access enabled from all networks (phase 4 of the learning path moves the vault behind a private endpoint). Add tags, then **Review + create** › **Create** › **Go to resource**. Copy the **Vault URI** from Overview into your notebook.

7b · The surprise: you are Owner, but the safe is locked

In the vault's left menu open **Objects** › **Secrets**. Instead of a list you will probably see a message like this:

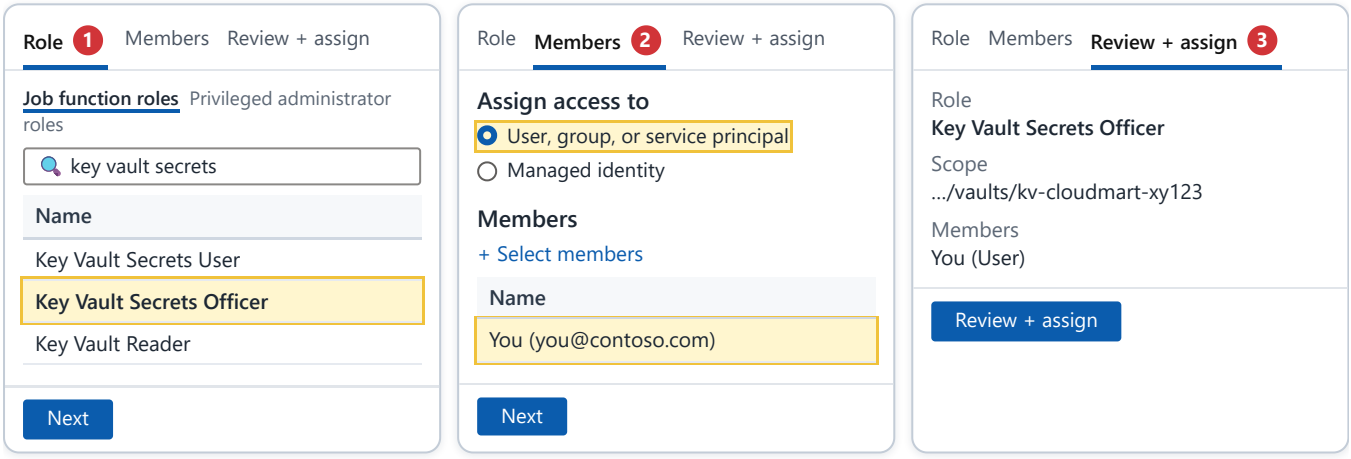


Simplified illustration. The exact wording varies; the meaning is "you have no data-plane role on this vault".

This is the **control plane vs data plane** idea from section 1.3. Owner let you *create* the vault; reading its secrets needs a separate **data** role. Give yourself one:

7c · Give yourself the data role

CLICK PATH **kv-cloudmart-xy123** › **Access control (IAM)** › **+ Add** › **Add role assignment**



Simplified illustration of the three-tab Add role assignment wizard. You will use this wizard about ten times in this guide.

1. **Role** tab: stay on **Job function roles**, search **key vault secrets**, select **Key Vault Secrets Officer** (read and write secrets), click **Next**.
2. **Members** tab: **Assign access to** = **User, group, or service principal**; click **+ Select members**, search your own name, click it, click **Select**, then **Next**.
3. Click **Review + assign** (it may ask twice). Wait 1–5 minutes, then refresh **Secrets**: the error is gone and the list is empty.

7d · Add the secret

CLICK PATH Secrets › + Generate/Import

Field	Enter or choose	Why
Upload options	`Manual`	
Name	`PaymentGateway--ApiKey`	Secret names can't contain ":". .NET turns "--" back into ":", so the app sees the setting `PaymentGateway:ApiKey`.
Secret value	any made-up text, e.g. `fake-key-123`	It is pretend. **Never** put a real key in a lab.
Set expiration date	tick it; about 90 days from today	An expiry is a promise to rotate the secret. Bicep sets 90 days too.
Enabled	`Yes`	

Click **Create**. The secret appears in the list with status Enabled.

7e · Let the two apps read it

Open **Access control (IAM)** › **+ Add** › **Add role assignment** again, this time for the identities from Step 6:

1. **Role**: **Key Vault Secrets User** (read only), **Next**.
2. **Members**: **Assign access to** = **Managed identity** › **+ Select members** › **Managed identity** = **User-assigned managed identity** › tick `id-cloudmart-lab-api` and `id-cloudmart-lab-func` › **Select**.
3. **Review + assign**.

Who	Role on this vault	Can do
You	Key Vault Secrets Officer	create, read, update and delete secrets
id-cloudmart-lab-api	Key Vault Secrets User	read secrets only
id-cloudmart-lab-func	Key Vault Secrets User	read secrets only

CHECK IT WORKED

Access control (IAM) › **Role assignments** tab lists these three entries (plus any inherited from the subscription, such as your Owner role).

WATCH OUT: ROLE ASSIGNMENTS ARE NOT INSTANT

New role assignments can take a few minutes, occasionally up to 10, to take effect. If something says "forbidden" right after you assigned a role, wait, refresh, or sign out and back in.

SAME THING IN BICEP

```
resource vault 'Microsoft.KeyVault/vaults@2024-11-01' = {
  properties: {
    sku: { family: 'A', name: 'standard' }
    enableRbacAuthorization: true      // "Azure role-based access control"
    softDeleteRetentionInDays: 7       // "Days to retain deleted vaults"
  }
}
// main.bicep passes the role list: API + Functions get keyVaultSecretsUser, you get
keyVaultSecretsOfficer
```

STEP 8

Storage account: product images and app files

TIME 15 min **COST** cents **BICEP** infra/modules/storage.bicep

WHY

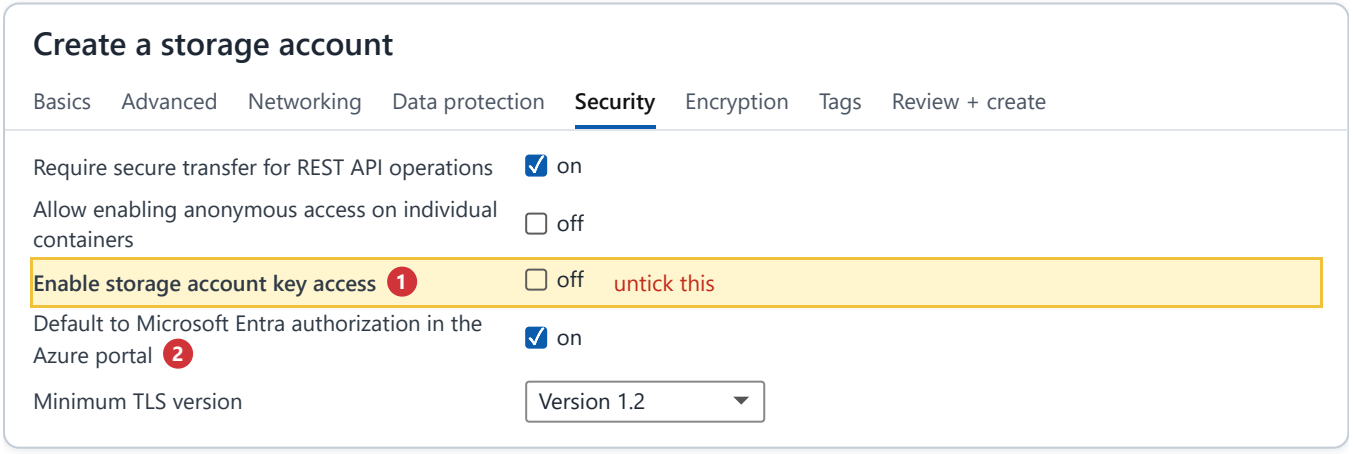
Cheap, durable storage for files ("blobs"). CloudMart keeps uploaded product images here, and the Function App keeps its own working files here. You will also switch off storage **keys**, so every caller must use Entra ID.

8a · Create the account

CLICK PATH Search › Storage accounts › + Create

Tab · field	Enter or choose	Why
Basics · Storage account name	`stcloudmartxy123`	3–24 **lowercase letters and digits only** (no hyphens), unique worldwide.
Basics · Region	`East US 2`	
Basics · Preferred storage type	`Azure Blob Storage or Azure Data Lake Storage Gen 2`	Optional; it only tailors the defaults shown.
Basics · Performance	`Standard`	Premium is for very low latency; not needed.
Basics · Redundancy	`Locally-redundant storage (LRS)`	Cheapest: 3 copies in one datacentre. Phase 11 explores ZRS and GRS.
Advanced · Access tier	`Hot`	For files read often.
Networking · Public network access	Enable from all networks	The shopper's browser uploads straight to it (Step 9).
Data protection	tick **soft delete for blobs** (7 days), **soft delete for containers** (7 days), **versioning for blobs**	A recycle bin and history for files. Same as Bicep.

Then the **Security** tab (on some screens these options sit on the Advanced tab):



Simplified illustration of the storage security settings.

1 **Turn key access off.** A storage key is a master password for the whole account: anyone who has it can do anything, and it never expires. With keys off, every caller (you, the apps, the portal) must sign in with Entra ID and have a data role. Bicep: `allowSharedKeyAccess: false`.

2 **Use Entra in the portal**, so the portal's file browser signs in as you instead of using a key.

Add tags, **Review + create** › **Create** › **Go to resource**. Open **Settings** › **Endpoints** and copy the **Blob service** URL (`https://stcloudmartxy123.blob.core.windows.net/`) into your notebook.

8b · Give yourself data access first

Same lesson as the vault: with keys off, even the portal needs **you** to hold a data role before it can list or create containers.

Access control (IAM) › **+ Add** › **Add role assignment** › role **Storage Blob Data Contributor** › member: **you** › **Review + assign**. Wait a minute or two.

8c · Create four containers

A **container** is a folder inside the account. Open **Data storage** › **Containers** › **+ Container** (or **+ Add container**) and create each of these with **Anonymous access level** = **Private**:

Container	What CloudMart keeps in it
`product-uploads`	images just uploaded by an admin, waiting to be processed
`product-images`	processed product images the shop displays
`function-deployments`	the Function App's code package
`eventgrid-deadletter`	upload events that failed (used from phase 5)

8d · Give the apps their storage roles

Repeat the Add role assignment wizard (one role per pass; you can pick both identities when they share a role). Choose **Assign access to** = **Managed identity**.

Identity	Role	Why it needs it
id-cloudmart-lab-api	Storage Blob Data Contributor	read and write product images
	Storage Blob Delegator	sign short-lived upload links (a "user delegation SAS") for the browser
id-cloudmart-lab-func	Storage Blob Data Owner	the Functions host's own files, and image processing
	Storage Queue Data Contributor	the Functions host's internal queues
	Storage Table Data Contributor	the Functions host's bookkeeping tables

WHAT IS A SAS?

A **shared access signature** is a link that grants narrow, temporary access, such as "upload this one file in the next 10 minutes". The API signs one with its identity so a shopper's browser can upload an image directly, without ever holding a key. This is called the **valet key** pattern.

CHECK IT WORKED

Containers shows four private containers. **Access control (IAM)** › **Role assignments** shows six storage roles: one for you, two for the API identity, three for the Functions identity.

SAME THING IN BICEP

```
properties: {
  allowSharedKeyAccess: false           // "Enable storage account key access": off
  allowBlobPublicAccess: false          // no anonymous containers
  defaultToOAuthAuthentication: true    // "Default to Microsoft Entra authorization"
  minimumTlsVersion: 'TLS1_2'
}
var containerNames = ['product-uploads', 'product-images', 'function-deployments', 'eventgrid-deadletter']
```

STEP 9

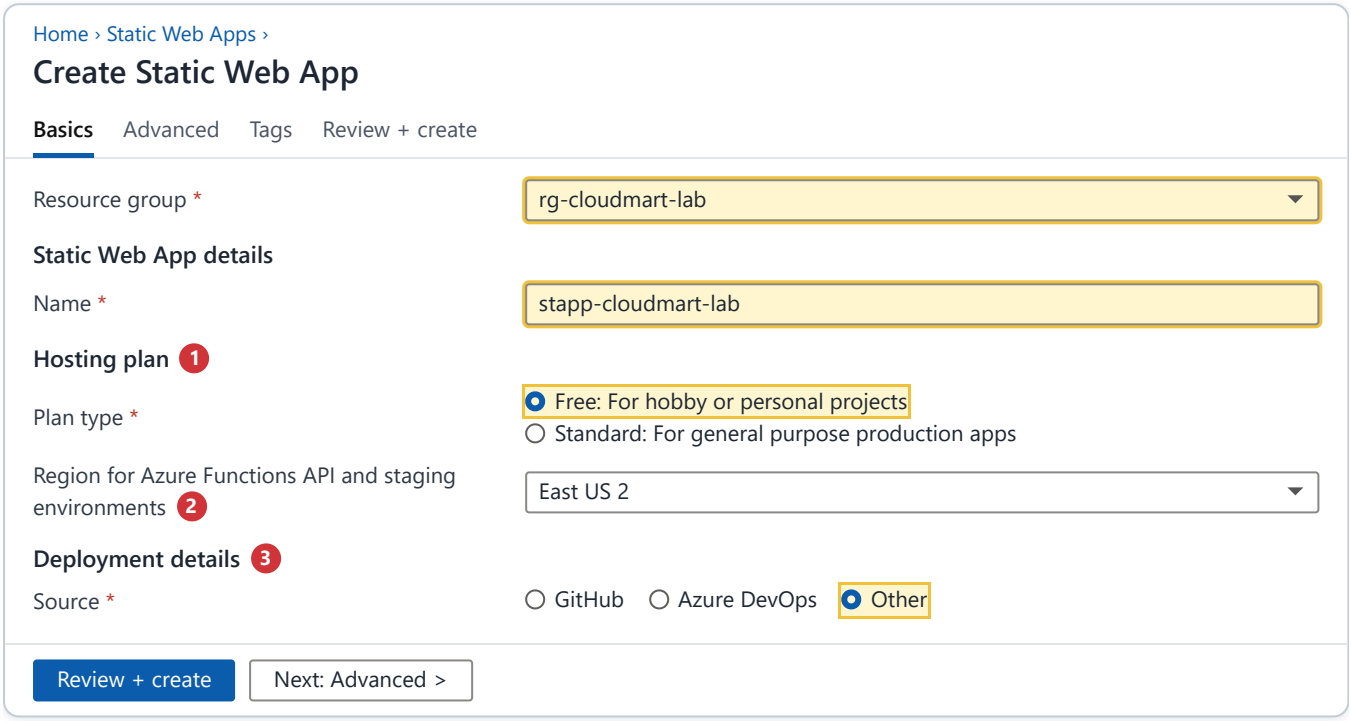
Static Web App: the shop's website

TIME 5 min **COST** free (Free plan) **BICEP** infra/modules/staticwebapp.bicep

WHY

CloudMart's website is a React app: just HTML, JavaScript and CSS files. Static Web Apps serves such files from Microsoft's global network, with HTTPS, for free. It is created before the API because the API and storage need to know its address.

CLICK PATH **Search** › **Static Web Apps** › **+ Create**



Simplified illustration.

1 **Plan type: Free.** Standard (about US$9 a month) is only needed in phase 10, to lock the site behind Front Door.

2 Static Web Apps is offered in only a few regions; the files themselves are served worldwide anyway.

3 **Source: Other.** You will upload the files yourself in Part 3. Choosing GitHub would create a pipeline in your repository.

Review + create › **Create** › **Go to resource**. Copy the **URL** from Overview (something like `https://<random-words>.azurestaticapps.net`) into your notebook. Opening it now shows a placeholder page.

9b · Let the website upload to storage (CORS)

Browsers block a web page from sending data to a different website unless that site says "I trust this origin". This rule is called **CORS**. Tell the storage account to trust your shop, then click **Save**:

CLICK PATH **stcloudmartxy123** › **Settings** › **Resource sharing (CORS)** › **Blob service**

Field	Enter	Why
Allowed origins	your Static Web App URL, e.g. `https://…azurestaticapps.net`	Exactly as copied: `https://`, no slash at the end.
Allowed methods	`PUT`, `OPTIONS`	Upload, plus the browser's "may I?" check.
Allowed / exposed headers	`*` and `*`	Any header.
Max age	`3600`	The browser remembers the answer for an hour.

STEP 10

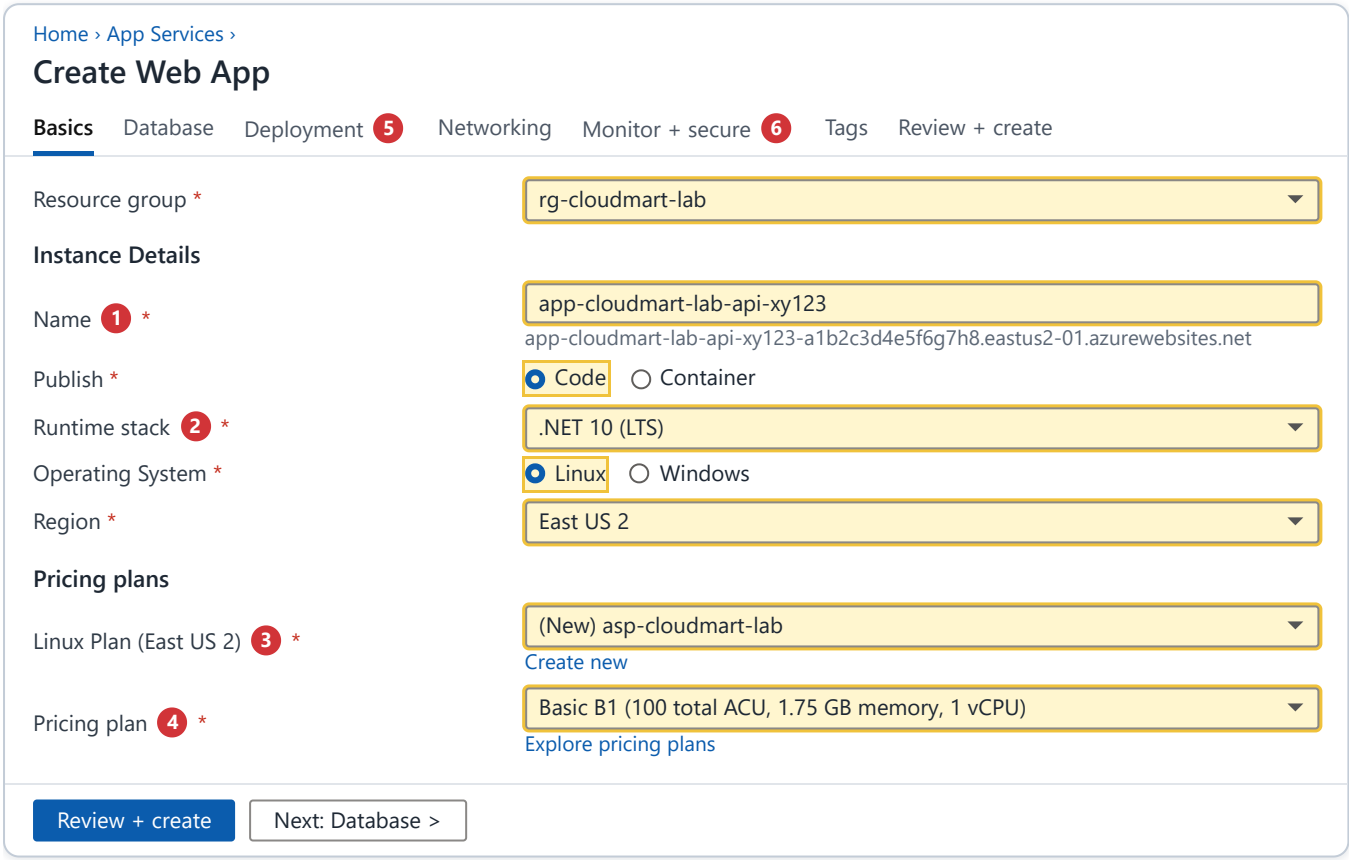
Web App: a home for the API

TIME 15 min | **COST** ≈ US$13/month (B1) | **BICEP** infra/modules/appservice.bicep

WHY

App Service runs web applications without you managing servers. The **App Service plan** is the machine you rent (size B1); the **Web App** is the application running on it. CloudMart's .NET API lives here.

CLICK PATH **Search** › **App Services** › **+ Create** › **Web App**



Simplified illustration of the Web App Basics tab.

1 **Name** must be unique. New apps get a **secure unique default hostname** with a random part and the region in it, which protects against "subdomain takeover". Use the exact address shown on Overview later.

2 **Runtime stack: .NET 10 (LTS)**, Operating System **Linux**: what CloudMart's API targets (`DOTNETCORE|10.0` in Bicep).

3 **Linux Plan**: **Create new** › `asp-cloudmart-lab` .

4 **Pricing plan: Basic B1.** If you only see Free or Premium, click **Explore pricing plans**. B1 is the cheapest plan with *Always On*; deployment slots and autoscale need Standard or above.

5 **Deployment** tab: Continuous deployment **Disable**; **Basic authentication: Disable** (code is uploaded with your Entra ID login, not a username and password).

6 **Monitor + secure** tab: **Enable Application Insights: Yes** › choose `appi-cloudmart-lab` . This adds the connection string to the app for you.

Leave **Database** and **Networking** at their defaults (public access on). Add tags, **Review + create** › **Create**. It takes a minute or two.

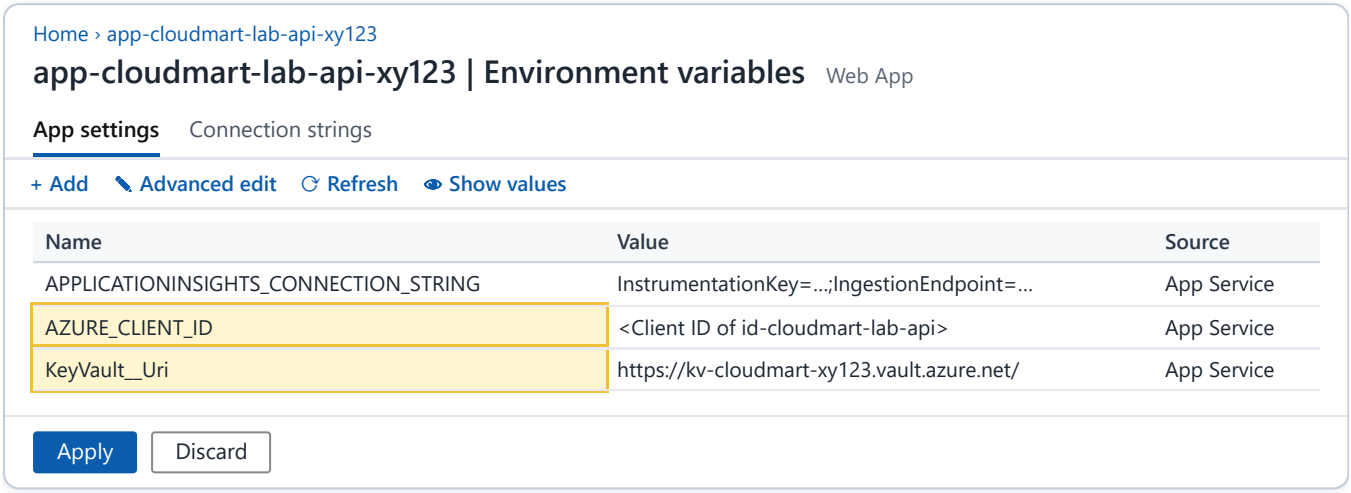
10b · Put the badge on the app

CLICK PATH app-cloudmart-lab-api-xy123 › Settings › Identity › User assigned › + Add

Pick `id-cloudmart-lab-api` and click **Add**. Leave **System assigned** off.

10c · App settings: tell the code where things are

App settings are name = value pairs the code reads at start-up, like a configuration file kept by Azure. Open **Settings** › **Environment variables** › **App settings** and click **+ Add** for each row:



Simplified illustration. Click **Apply** at the bottom when all rows are in, then **Confirm**; the app restarts.

Name	Value	What it does
`AZURE_CLIENT_ID`	its Client ID (Step 6)	Which badge the code should wear.
`CLOUDMART_REGION`	`eastus2`	Shown on the shop's Platform page.
`KeyVault__Uri`	Vault URI (Step 7)	The API loads its secrets from here at start-up.
`Storage__BlobEndpoint`	Blob service URL (Step 8)	Where product images live.
`Cors__AllowedOrigins`	Static Web App URL `,http://localhost:5173`	Which websites may call the API (comma-separated, no spaces).
`Auth__AllowAnonymousDevMode`	`true`	Lab only: everyone is a demo admin (phase 6 adds sign-in).
`APPLICATIONINSIGHTS_CONNECTION_STRING`	already there	Added by the wizard; just check.

"__" stands for ":" because Linux settings can't contain a colon: .NET reads `KeyVault__Uri` as `KeyVault:Uri`.

10d · Safety settings

1. **Settings** › **Configuration** › **General settings**: **HTTPS Only** On · **Always on** On · **HTTP version** 2.0 · **Minimum inbound TLS version** 1.2 · **SCM** and **FTP Basic Auth Publishing Credentials** Off · **FTP state** Disabled › **Save**.
2. **Monitoring** › **Health check** › **Enable**, path `/health/live` › **Save**. App Service now restarts the app if it stops answering.

CHECK IT WORKED

On **Overview**, copy the **Default domain** into your notebook and open it with `https://` in front. You see a default "waiting for your content" page: the server works; the code comes in Part 3.

STEP 11

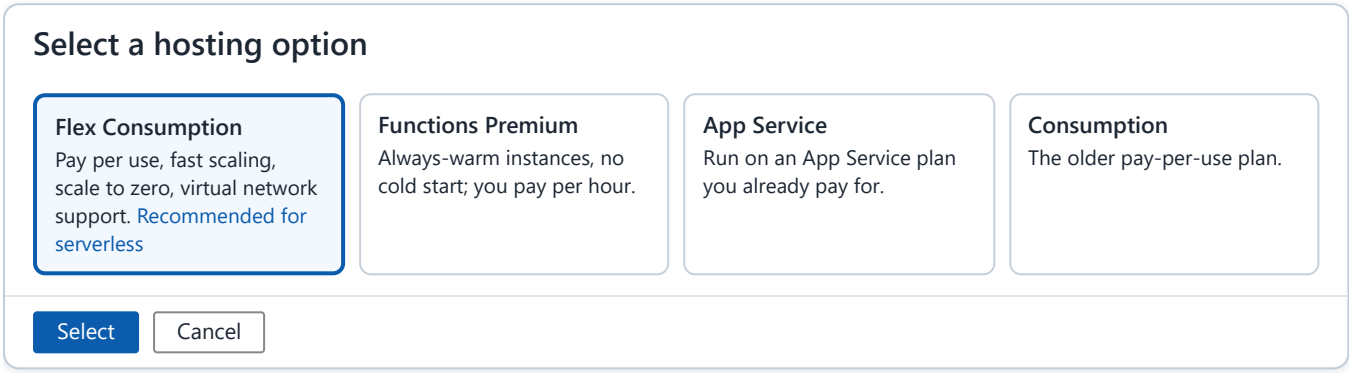
Function App: background jobs, serverless

TIME 10 min | **COST** ≈ $0 idle | **BICEP** infra/modules/functionapp.bicep

WHY

Azure Functions run small pieces of code when something happens: a message arrives, a file is uploaded, a timer ticks. On the **Flex Consumption** plan you pay only while code runs, and it scales to zero when idle. CloudMart's order processing and image handling run here.

CLICK PATH Search › Function App › + Create › Flex Consumption › Select



Simplified illustration. The choices offered may differ; pick **Flex Consumption**.

Tab · field	Enter or choose	Why
Basics · Function App name	`func-cloudmart-lab-xy123`	Unique worldwide; write it in your notebook.
Basics · Region	`East US 2`	Flex isn't in every region; unsupported ones are hidden.
Basics · Runtime stack · Version	`.NET` · `10 (isolated worker model)`	Matches CloudMart's Functions project.
Basics · Instance size	`2048 MB`	Memory per instance; the recommended default.
Storage · Storage account	`stcloudmartxy123`	Use the account from Step 8 instead of creating a new one.
Networking · Enable public access	On	Private networking comes in phase 2.
Monitoring · Enable Application Insights	Yes › `appi-cloudmart-lab`	Same monitoring as the API.
Authentication · Authentication type	`Managed identity`	**Required here.** You turned storage keys off in Step 8, so the app must reach its storage with an identity.

WATCH OUT: WHICH IDENTITY?

On the Authentication tab the portal may create a **new** user-assigned identity for the app, and give it the storage roles it needs. That is fine. If it lets you choose an existing one, pick `id-cloudmart-lab-func`. Either way, after creation open **Settings** › **Identity** › **User assigned** and make sure `id-cloudmart-lab-func` is attached (**+ Add** if not): it is the badge with the Key Vault role from Step 7.

Add tags, **Review + create** › **Create** › **Go to resource**.

11b · App settings for the Function App

Settings › **Environment variables** › **App settings** › **+ Add** for each row, then **Apply** › **Confirm**. Leave the `AzureWebJobsStorage__…` and Application Insights settings the wizard created as they are.

Name	Value	What it does
`AZURE_CLIENT_ID`	Client ID of `id-cloudmart-lab-func`	Which badge the code wears.
`CLOUDMART_REGION`	`eastus2`	
`Storage__BlobEndpoint`	your Blob service URL	Where images are processed.
`AzureWebJobs.FulfillOrder.Disabled`	`true`	These functions listen to Service Bus, Event Hubs or a database, which you add in phases 4 and 5. Switching them off stops them failing at start-up. Bicep does exactly this.
`AzureWebJobs.FlagHighValueOrder.Disabled`	`true`	
`AzureWebJobs.NotifyCustomer.Disabled`	`true`	
`AzureWebJobs.AggregateProductViews.Disabled`	`true`	
`AzureWebJobs.DailyOrderReport.Disabled`	`true`	

OPTIONAL, ADVANCED: A KEY VAULT REFERENCE

Bicep also gives the Function App the setting `PaymentGateway__ApiKey` with the value `@Microsoft.KeyVault(VaultName=kv-cloudmart-xy123;SecretName=PaymentGateway--ApiKey)`. Azure then fetches the secret itself and hands it to the code as a normal setting. With a user-assigned identity, Azure must be told which badge to use for this, and the portal has no simple button for it at the time of writing; it takes one CLI command:

```
az functionapp update -g rg-cloudmart-lab -n func-cloudmart-lab-xy123 \
  --set keyVaultReferenceIdentity=<resource ID of id-cloudmart-lab-func>
```

(Find the resource ID on the identity's **Properties** page.) A good example of why the CLI and Bicep exist: not everything has a button.

CHECK IT WORKED

Overview shows Status **Running** and an empty **Functions** list. The functions appear after you deploy the code in Part 3.

SAME THING IN BICEP

```
resource plan 'Microsoft.Web/serverfarms@2024-04-01' = {
  sku: { tier: 'FlexConsumption', name: 'FC1' }
}
functionAppConfig: {
  scaleAndConcurrency: { maximumInstanceCount: 40, instanceMemoryMB: 2048 }
  runtime: { name: 'dotnet-isolated', version: '10.0' }
}
// app settings: AzureWebJobsStorage__credential = 'managedidentity' (no connection string)
```

STEP 12

An alert that emails you

TIME 10 min **COST** ≈ US$1.50/month **BICEP** infra/modules/monitoring.bicep

WHY

You want to hear about errors before your users do. This alert runs a log query every 5 minutes and emails you if the API throws more than 5 errors (exceptions). An **action group** is the list of who to tell and how.

CLICK PATH Search › Monitor › Alerts › + Create › Alert rule

Tab · field	Enter or choose	Why
Scope	`appi-cloudmart-lab` › Apply	The query runs against this app's data.
Condition · Signal name	`Custom log search`	An alert based on a KQL query.
Condition · Search query	`exceptions | where client_Type != "Browser"` › **Run** › **Continue editing alert** Counts server-side errors only, not errors in shoppers' browsers.	
Measure · Aggregation type	`Table rows` · `Count`	Count the matching rows.
Aggregation granularity	`5 minutes`	Each check looks at 5 minutes of data.
Operator · Threshold value	`Greater than` · `5`	More than 5 errors in 5 minutes.
Frequency of evaluation	`5 minutes`	How often to check. More often costs more.
Actions	+ **Create action group** (below)	
Details · Severity	`2 - Warning`	0 is critical, 4 is verbose.
Details · Alert rule name	`CloudMart lab: exception spike`	Bicep's is "CloudMart: exception spike".
Details · Region	`East US 2`	

Create the action group (from the Actions tab)

1. **Basics**: resource group `rg-cloudmart-lab`, **Action group name** `ag-cloudmart-lab`, **Display name** `cloudmart` (max 12 characters; it appears in messages). Region: Global.
2. **Notifications**: type **Email/SMS message/Push/Voice**, name `owner`; tick **Email** and enter your address › **OK**.
3. **Review + create** › **Create**. You return to the alert rule with the group selected.

Finish the alert rule with **Review + create** › **Create**.

CHECK IT WORKED

You get an email saying you were added to the action group *cloudmart*. **Monitor** › **Alerts** › **Alert rules** lists your rule as Enabled. In Part 3 you can trigger it on purpose from CloudMart's Admin page (the "chaos" buttons).

STEP 13

Guard rails: a policy and a lock

TIME 10 min **COST** free **BICEP** infra/modules/governance.bicep

WHY

RBAC (roles) controls *who* can act. **Azure Policy** controls *what* may exist, whoever asks. **Locks** protect against accidents, even by Owners.

13a · Policy: only allow East US 2

CLICK PATH Search › Policy › Authoring › Assignments › Assign policy

Tab · field	Enter or choose	Why
Basics · Scope	… › your subscription › `rg-cloudmart-lab` › Select	The rule applies only inside this group.
Basics · Policy definition	… › search `Allowed locations` › Add	A built-in rule from Microsoft.
Basics · Assignment name	`CloudMart lab: allowed locations`	
Basics · Policy enforcement	`Enabled`	"Disabled" would only report, not block.
Parameters · Allowed locations	`East US 2`	Untick "Only show parameters that need input" if the list is hidden. Add your Static Web App's region if it differs.
Non-compliance messages	`CloudMart lab resources must be in East US 2.`	Shown to whoever gets blocked.

Review + create › **Create**. A new assignment can take up to 30 minutes to start working.

CHECK IT WORKED (TRY TO BREAK IT)

Later, start creating any small resource (for example a storage account) in `rg-cloudmart-lab` but choose region **West Europe**. **Review + create** fails validation with `RequestDisallowedByPolicy` and your message. Cancel the wizard.

OPTIONAL: THE SECOND POLICY BICEP ASSIGNS

CloudMart's Bicep also assigns **Inherit a tag from the resource group if missing** with tag name `environment`. It uses the **Modify** effect, so it needs permission to change resources: on the **Remediation** tab, tick **Create a Managed Identity** (system assigned). The portal gives that identity the role it needs. Then any new resource without an `environment` tag gets the group's tag automatically.

13b · Lock: prevent accidental deletion

CLICK PATH rg-cloudmart-lab › Settings › Locks › + Add

Lock name `no-delete`, Lock type **Delete** (called `CanNotDelete` in the CLI and Bicep), notes "lab safety" › **OK**. Now try to delete the storage account: Azure refuses, even though you are Owner.

WATCH OUT

A **Read-only** lock is much stronger than its name suggests: it also blocks actions that read via a POST, such as listing keys or restarting an app. And you must **remove the lock before Part 5**, or the clean-up fails.

PART 2 DONE!

Open `rg-cloudmart-lab`. Its Overview lists everything you built (plus a few things Azure adds itself, such as the Function App's own plan and a *Failure Anomalies* alert). This is the phase 1 foundation of CloudMart.

PART 3

Put CloudMart's code on it

The portal is for building and inspecting. Code is uploaded with a command or a pipeline.

For this part you need a laptop with the **.NET 10 SDK**, **Node.js 24**, **Git**, the **Azure CLI** and a copy of the CloudMart repository. Open a terminal in the repository folder and sign in once:

```
az login                          # opens a browser; pick the same account as the portal
az account show -o table          # check it shows the subscription from Step 1
```

3.1 · The API → Web App

```
# Windows PowerShell
dotnet publish src/CloudMart.Api -c Release -o out/api
Compress-Archive -Path out/api/* -DestinationPath out/api.zip -Force
az webapp deploy -g rg-cloudmart-lab -n app-cloudmart-lab-api-xy123 --src-path out/api.zip --type zip
```

In the portal, open the Web App's **Default domain** and add `/health/live`, then `/api/platform/info`: you should get a short text or JSON reply instead of the placeholder page.

3.2 · The Functions → Function App

```
dotnet publish src/CloudMart.Functions -c Release -o out/func
Compress-Archive -Path out/func/* -DestinationPath out/func.zip -Force
az functionapp deployment source config-zip -g rg-cloudmart-lab -n func-cloudmart-lab-xy123 --src
out/func.zip
```

In the portal, the Function App's **Overview** › **Functions** list now shows CloudMart's functions; the five you switched off show as **Disabled**.

3.3 · The website → Static Web App

1. Build it: `cd src/cloudmart-web`, then `npm install` and `npm run build`. The result is in `dist/`.
2. Open `dist/config.json` in a text editor and fill in two values (leave the `entra` part empty until phase 6):

```
{
  "apiBaseUrl": "https://<your Web App default domain>",
  "entra": { "clientId": "", "tenantId": "", "apiScope": "" },
  "appInsightsConnectionString": "<connection string from Step 5>"
}
```

3. In the portal: Static Web App › **Overview** › **Manage deployment token** › copy the token.
4. Upload: `npx @azure/static-web-apps-cli deploy dist --env production --deployment-token <token>`

> **WATCH OUT: THE DEPLOYMENT TOKEN IS A SECRET**
>
> Anyone with it can replace your website. Don't paste it in chat or email, and don't save it in the repository. If it leaks, click **Reset token** on the same page.

3.4 · Try the shop

1. **Shop** — Open your Static Web App URL. Products load from the API.
2. **Order** — Place an order: it goes *Pending*, then *Confirmed* a moment later.
3. **Platform** — Shows **App Service**, region **eastus2**, and which parts are still in-memory.
4. **Admin** — Upload a product image: it lands in the `product-uploads` container.

Most parts still say *In-memory* on the Platform page. That is expected: phases 4 and 5 of the learning path swap them for Cosmos DB, Azure SQL and Service Bus.

3.5 · Watch it from the portal

Where in the portal	What you see
Storage account › **Storage browser** › Blob containers › `product-uploads`	The image you uploaded from Admin, sent straight from the browser with a SAS link.
Application Insights › **Application map**	A diagram of the browser, the API and its dependencies, drawn from real traffic.
Application Insights › **Live metrics**	Requests per second while you click around the shop.
Application Insights › **Transaction search**	One request followed end to end: browser → API → storage.
Application Insights › **Logs**	Run `requests | summarize count() by resultCode` to count responses by status.
Web App › **Log stream**	The API's console output, live. The first place to look if it won't start.
Key Vault › **Logs** or **Activity log**	Who did what to the vault and its settings.

TRIGGER YOUR ALERT

On the shop's **Admin** page, use the chaos buttons to make the API throw errors for a few minutes. Within about 10 minutes you should get the "exception spike" email from Step 12. That is the whole monitoring chain working: app → Application Insights → Log Analytics → alert rule → action group → you.

IF SOMETHING DOESN'T WORK

Website loads but shows no products: check `apiBaseUrl` in `config.json` and `Cors__AllowedOrigins` on the Web App (exact URL, `https://`, no trailing slash). API shows "Application Error": open **Log stream**; the usual cause is a wrong `KeyVault__Uri` or a missing role from Step 7e. See the Troubleshooting table at the back.

PART 4

From clicks to code

See that every click was really a request to Resource Manager, and then let Bicep do the clicking.

4.1 · See your clicks as Bicep

CLICK PATH rg-cloudmart-lab › Automation › Export template › Bicep

The portal writes Bicep (or ARM JSON) describing everything in the group. Find the key vault in it and compare it with `infra/modules/keyvault.bicep`: you will see the same `enableRbacAuthorization: true` and `softDeleteRetentionInDays: 7` you clicked. Exported templates are a learning aid, not production code: they are long and full of default values, which is why CloudMart's Bicep is written by hand.

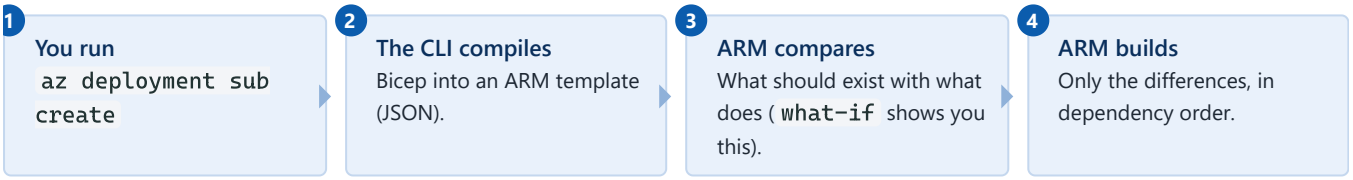
4.2 · Every click is recorded

Open the resource group's **Activity log**. Each create, role assignment and setting change is there, with who did it and when. This is the audit trail Azure keeps for every subscription (for 90 days).

4.3 · Every "Create" was a deployment

Open the resource group's **Settings** › **Deployments**. Each wizard you finished appears as a deployment, the same kind of object `az deployment sub create` creates for CloudMart. Click one and choose **Template** to see what the portal sent.

A Bicep deployment is the same thing, only bigger: one deployment that describes the whole environment instead of one resource.



Declarative and repeatable. Run it again and nothing changes; flip a switch in `main.bicepparam` and only that difference is applied. This is why CloudMart's 16 phases are switches in one file.

PORTAL WIZARD VS BICEP, SIDE BY SIDE

	A portal wizard	CloudMart's Bicep
Creates	one resource (sometimes two or three)	the whole environment, about 20 resources and their roles
Remembers your choices	no: next time you click again	yes: they are in Git, reviewed like code
Time for phase 1	about two hours of clicking	about five minutes
Best for	learning what the options mean	building it right, every time

4.4 · Portal ↔ CLI ↔ Bicep cheat sheet

CLI commands are shortened to show their shape; run `az <command> --help` for the full options.

What you did in the portal	Azure CLI (shortened)	Where in CloudMart's Bicep
Resource group (Step 2)	`az group create -n rg-cloudmart-lab -l eastus2 --tags …`	`main.bicep` : `rg`
Budget (3)	`az consumption budget create …`	`governance.bicep`
Workspace, App Insights (4, 5)	`az monitor log-analytics workspace create …` `az monitor app-insights component create …`	`monitoring.bicep`
Managed identity (6)	`az identity create -g … -n id-cloudmart-lab-api`	`identity.bicep`
Key Vault (7)	`az keyvault create … --enable-rbac-authorization true`	`keyvault.bicep`
Role assignment (7, 8)	`az role assignment create --assignee … --role … --scope …`	`roleAssignments` in each module
Storage, keys off (8)	`az storage account create … --allow-shared-key-access false`	`storage.bicep`
Static Web App (9)	`az staticwebapp create … --sku Free`	`staticwebapp.bicep`
Web App + plan (10)	`az appservice plan create … --is-linux --sku B1` `az webapp create … --runtime "DOTNETCORE:10.0"`	`appservice.bicep`
App settings (10, 11)	`az webapp config appsettings set … --settings A=B`	`apiSettings`, `functionsSettings` in `main.bicep`
Function App, Flex (11)	`az functionapp create … --flexconsumption-location eastus2`	`functionapp.bicep`
Alert, action group (12)	`az monitor scheduled-query create …` `az monitor action-group create …`	`monitoring.bicep`
Policy (13)	`az policy assignment create --policy … --scope …`	`governance.bicep`
Lock (13)	`az lock create -g … -n no-delete --lock-type CanNotDelete`	learning path, phase 1 lab

4.5 · Now do it the real way, with Bicep

You have built the foundation by hand in about two hours. CloudMart's Bicep builds the same thing, plus the pieces later phases need, in about five minutes, and can rebuild it identically any time.

1. **Clean up the lab first** (Part 5), so you don't pay twice and don't confuse the two copies.
2. **Find your IDs in the portal**: search **Microsoft Entra ID** › **Users** › select yourself. Copy your **Object ID** and **User principal name**.
3. **Fill in** `infra/main.bicepparam`: `deployerPrincipalId` = your Object ID, `deployerPrincipalName` = your user principal name, `ownerEmail` = where alerts go.
4. **Preview, then deploy** from a terminal (your laptop, or **Cloud Shell** in the portal after cloning the repository there):

```
az deployment sub what-if -l eastus2 -f infra/main.bicep -p infra/main.bicepparam
az deployment sub create  -l eastus2 -f infra/main.bicep -p infra/main.bicepparam
```

5. **Watch it in the portal** while it runs: **Subscriptions** › your subscription › **Settings** › **Deployments** › the newest deployment. Each module (governance, monitoring, identity, key-vault, storage …) appears as it starts and finishes.
6. **Explore the result**: open `rg-cloudmart-dev`. Every resource you met in Part 2 is there, with the same settings. Use your new portal skills to inspect it, but change things through the Bicep files, not by clicking.

> **WATCH OUT: `FALSE` DOES NOT DELETE**
>
> Setting a switch back to `false` and running `az deployment sub create` stops *managing* that resource, but does not delete it. Delete it yourself, delete the resource group, or use a **deployment stack** (learning path, phase 8), which really removes what you take out of the template.

> **THE HABIT TO KEEP FOR EVERY PHASE**
>
> The learning path's loop is: **click it** once in the portal to see the options, **script it** once with the CLI to learn the model, then **read the Bicep** that does it. This guide was the "click it" half for phase 1.

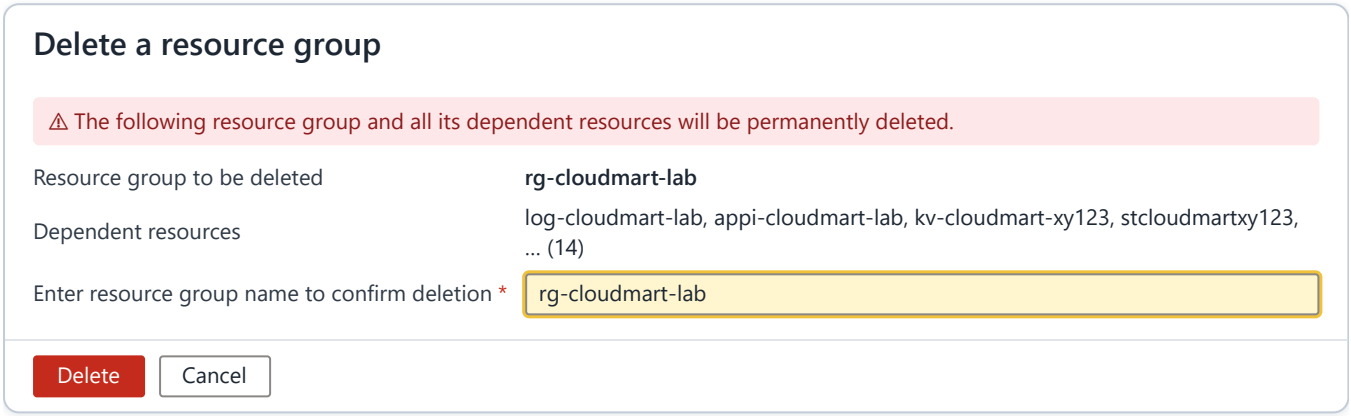
PART 5

Clean up and reference

Delete everything, then keep the troubleshooting table and glossary handy.

Clean up: delete everything

1. **Remove the lock**
   rg-cloudmart-lab › Locks › delete `no-delete`.
2. **Delete the group**
   Overview › Delete resource group › type its name.
3. **Purge the vault**
   Only if you want to reuse its name now.
4. **Check tomorrow**
   Cost Management shows spending stopped.



Simplified illustration. Typing the name is Azure's "are you sure?". Deletion takes a few minutes; watch the bell.

The Key Vault comes back to haunt you (on purpose)

Deleted vaults are **soft-deleted**: kept for 7 days (the number you chose in Step 7), so they can be recovered. While it waits, nobody can create a vault with that name. To free the name now:

CLICK PATH **Search** › **Key vaults** › **Manage deleted vaults** › **your subscription** › **kv-cloudmart-xy123** › **Purge**

What else to check

- **Budgets** and **Policy assignments** scoped to the resource group normally go with it. Check **Cost Management** › **Budgets** and **Policy** › **Assignments**; delete anything left over.
- If the Function App wizard created an extra storage account or identity outside the group, delete it too (search *All resources* for "cloudmart").
- The deployment token and connection strings you copied stop working when the resources are deleted. Delete your notes anyway.

CHECK IT WORKED

Resource groups no longer lists `rg-cloudmart-lab`. The next day, **Cost Management** › **Cost analysis** shows no new cost for it.

Troubleshooting: messages and fixes

What you see	What it means	How to fix it
"The subscription is not registered to use namespace 'Microsoft....'"	That service isn't switched on for your subscription yet.	Subscription › **Resource providers** › find the namespace › **Register**; wait for *Registered*; retry.
Name "is already taken" / "is not available"	Vault, storage, web and function app names are global: someone has it.	Change the digits in your suffix.
"This region has quota of 0 instances for your subscription"	New and free subscriptions have small limits for some plans.	Try another region (and add it to your policy), or request more quota under **Quotas** / **Help + support**.
`RequestDisallowedByPolicy`	Your policy from Step 13 blocked the region (or another rule did).	Use East US 2, or edit the assignment's **Parameters**.
Key Vault: "The operation is not allowed by RBAC" or "You are unauthorized to view these contents"	You have a control-plane role (Owner) but no data role.	Give yourself **Key Vault Secrets Officer** (Step 7c). Wait up to 10 minutes; refresh or sign in again.
Storage: "You do not have permissions to list the data using your user account" or "Key based authentication is not permitted"	Keys are off (good!) and you have no data role.	Give yourself **Storage Blob Data Contributor** (Step 8b). Wait a few minutes.
Function App shows storage errors or "runtime is unreachable"	It can't reach its storage account: it's trying to use a key, or its identity lacks roles.	Use **Managed identity** authentication (Step 11); check the identity has the three storage roles (Step 8d).
Web App shows "Application Error" or HTTP 500.30	The API crashed while starting.	Open **Log stream**. Usual causes: wrong `KeyVault__Uri`, missing `AZURE_CLIENT_ID`, or the identity's vault role not yet active.
Web App shows "waiting for your content"	No code deployed yet.	Part 3.1.
Shop loads but no products; browser console mentions CORS	The API doesn't trust the website's address, or the website calls the wrong address.	Check `Cors__AllowedOrigins` (Step 10c) and `apiBaseUrl` (Part 3.3): exact, `https://`, no trailing slash.
Image upload fails in Admin	Storage CORS or the API's storage roles are missing.	Step 9b (CORS) and Step 8d (Blob Data Contributor + Blob Delegator).
"...cannot perform delete operation because following scope(s) are locked" (`ScopeLocked`)	Your lock from Step 13.	Remove the lock first.
Can't create a vault: name "is already in use" after you deleted it	It is soft-deleted.	Purge it (Part 5) or pick another name.
A role "doesn't work" right after assigning it	Role assignments take a few minutes to spread.	Wait 5–10 minutes; sign out and in.

WHERE AZURE TELLS YOU MORE

Failed deployments: resource group › **Deployments** › the failed one › **Error details**. Anything that changed: **Activity log**. A resource misbehaving: its **Diagnose and solve problems** page, which often names the cause.

Glossary

Action group: a list of people and channels (email, SMS, webhook) an alert notifies.

App Service plan: the rented machine(s) your web apps run on; you pay for it per hour.

ARM (Azure Resource Manager): the service behind the portal, CLI and Bicep that checks and carries out every request.

Bicep: Microsoft's language for describing Azure resources in files. Compiles to ARM templates (JSON).

Blade: a page in the portal, such as a resource's Overview.

Blob: a file stored in a storage account container.

Cloud Shell: a terminal in the portal with the Azure CLI and Bicep preinstalled.

Control plane: managing a resource (create, configure, delete). Roles: Owner, Contributor, Reader.

CORS: a browser rule; a site must say which other websites may call it.

Data plane: using what is inside a resource (secrets, files, messages). Needs data roles.

Deployment: a record of a request to create or update resources, from a wizard, CLI or Bicep.

Drift: when a resource is changed by hand and no longer matches its Bicep.

Entra ID (Microsoft Entra ID): Azure's directory of users, groups and app identities. Formerly Azure Active Directory.

Flex Consumption: the serverless plan for Azure Functions: pay per use, scale to zero.

Idempotent: running it twice gives the same result as running it once. Bicep deployments are.

KQL (Kusto Query Language): the query language for Log Analytics and Application Insights.

Lock: stops a resource being deleted (Delete) or changed (Read-only), even by Owners.

Managed identity: an identity Azure manages for an app, so the app needs no password or key.

Policy (Azure Policy): rules about what may exist, e.g. "only in East US 2". Effects include Deny, Audit and Modify.

RBAC (role-based access control): giving a role to someone (or something) at a scope.

Region: a set of datacentres in one geography, e.g. East US 2.

Resource: one thing you create: a vault, an app, a storage account.

Resource group: a folder of resources that share a lifecycle.

Resource provider: the Azure service that runs a resource type, e.g. `Microsoft.KeyVault`.

Role assignment: role + who + scope, e.g. "Key Vault Secrets User for id-…-api on this vault".

SAS (shared access signature): a link that grants narrow, temporary access to storage.

Scope: where a role, policy or lock applies: management group, subscription, resource group or resource.

SKU / pricing tier: the size or edition of a resource, e.g. B1, Standard, Free.

Soft delete: deleted things are kept for a while so they can be recovered.

Subscription: the billing and access boundary that resources belong to.

Tag: a name = value label on a resource.

Tenant: your organisation's instance of Microsoft Entra ID.

What-if: a preview of what a Bicep deployment would change, before it changes anything.

What's next

You now know your way around the portal and have seen every building block of CloudMart's phase 1. Each later phase of `docs/learning-path.md` turns on more of Azure with a switch in `infra/main.bicepparam`. Here is where each one shows up in the portal:

Phase	Switch it on in Bicep	Then look in the portal at
2 · Networking	`deployNetwork`, `deployJumpbox`	Virtual network › Subnets; Network security groups; Bastion › Connect to the jumpbox
3 · Compute	`deployContainerApps`, `deployAks`	Container App › Revisions and replicas; Kubernetes service › Workloads
4 · Data	`deployCosmos`, `deploySql`	Cosmos DB › Data Explorer (request charges); SQL database › Query editor
5 · Messaging	`deployMessaging`	Service Bus › Topics › Service Bus Explorer; Event Hubs › Metrics
6 · Security	`enableEntraAuth`	Microsoft Entra ID › App registrations › App roles
7 · Observability	`deployGrafana`	Application Insights › Workbooks; Monitor › Alerts
8 · DevOps	pipeline, `deployAcr`	Container registry › Repositories; subscription › Deployments by the pipeline
9 · Governance	`deployGovernance` (on)	Policy › Compliance; Cost Management › Cost analysis by tag
10 · Architecture	`deployFrontDoor`	Front Door › Origin groups; WAF policy › Managed rules
11 · Continuity	`sqlSecondaryLocation`, …	SQL server › Failover groups › Failover
12 · Application integration	`deployAppConfig`, `deployRedis`, `deployApim`	App Configuration › Feature manager; Azure Managed Redis › Metrics; API Management › APIs › Test
13 · Hybrid networking	`deployFirewall`; hybrid lab: `deployOnPremSim`, `deployVpnGateway`, `deployDnsResolver`	Firewall policy › Application rules; Virtual network gateway › Connections; DNS private resolver › Inbound endpoints
14 · Data integration and analytics	`deployAnalytics`, `enableClickstreamCapture`	Data factory › Launch studio › Monitor › Pipeline runs; the data lake's storage account › Containers
15 · Migration	`deployMigrationLab`, `migrateProjectLocation`	Azure Migrate › Servers, databases and web apps › Assess; the legacy VM › Run command
16 · Microservices	`deployPayments`, `deployShipping`	Storage account › Storage browser › Tables (saga history); Application Insights › Transaction search

SOME SWITCHES ARE EXPENSIVE

AKS, Application Gateway, Front Door Premium, Bastion Basic, Azure Firewall, the SQL failover group, the VPN gateways (about US$187 a month), the DNS Private Resolver (about US$180 a month) and Event Hubs Capture (about US$73 a month) cost real money every hour. The learning path flags each one. Turn it on, learn, turn it off (or delete) the same day.

KEEP THIS GUIDE'S THREE HABITS

1. **Budget first, delete after.** Never leave a lab running "just in case".
2. **Remember the two locks.** Owner opens the building, a data role opens the safe.
3. **Click to learn, Bicep to build.** Look at everything in the portal; change things in the code.

This guide follows the CloudMart repository's `infra/main.bicep` with its default switches. Portal screens are simplified illustrations based on Microsoft Learn documentation as of October 2026; Microsoft updates the portal often, so a label or tab may differ slightly from what you see. When in doubt, the search bar and the **?** help icon in the portal are your friends.